

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(УНИВЕРСИТЕТ ИТМО)

Факультет «Систем управления и робототехники»

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ №1

По дисциплине «Теория автоматического управления»
на тему:
«УПРАВЛЯЕМОСТЬ И НАБЛЮДАЕМОСТЬ(17 вариант)»

Студент:
Шишкина М.Н., ТеорАвтоУп R23 бак 1.1.3

Проверил:
Пашенко Артем Витальевич, ассистент

г. Санкт-Петербург
2026

СОДЕРЖАНИЕ

1	Исследование управляемости	3
1.1	Задание	3
1.2	Ход работы	3
1.3	Выводы	12
2	Еще одно исследование управляемости	12
2.1	Задание	12
2.2	Ход работы	13
2.3	Выводы	19
3	Исследование наблюдаемости	19
3.1	Задание	19
3.2	Ход работы	20
3.3	Выводы	27
4	Еще одно исследование наблюдаемости	27
4.1	Задание	27
4.2	Ход работы	28
4.3	Выводы	34
5	Исследование управляемости по выходу	35
5.1	Задание	35
5.2	Ход работы	35
5.3	Выводы	38
6	Вывод	39
	Приложение А	40
	Приложение Б	46
	Приложение В	53
	Приложение Г	59
	Приложение Д	69

1 Исследование управляемости

1.1 Задание

Система описывается уравнением состояния:

$$\dot{x} = Ax + Bu$$

где:

$$A = \begin{bmatrix} 1 & 2 & -1 \\ -6 & -7 & -2 \\ 6 & 6 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} -2 \\ 4 \\ -2 \end{bmatrix}, \quad x_1 = \begin{bmatrix} 2 \\ -3 \\ 3 \end{bmatrix}$$

Необходимо:

1. Исследовать управляемость системы:
 - а) Найти матрицу управляемости системы, определить её ранг, сделать вывод об управляемости системы в целом.
 - б) Найти собственные числа матрицы A , найти для каждого из собственных чисел матрицу Хаутуса (для управляемости), определить ранги матриц, сделать выводы об управляемости каждого собственного числа и системы в целом.
 - в) Найти вещественную Жорданову форму системы и сделать выводы об управляемости каждого собственного числа и системы в целом.
 2. Найти Граммиан управляемости системы относительно времени $t_1 = 3$, вычислить его собственные числа. Проанализировать полученные собственные числа с точки зрения управляемости системы.
 3. Найти управление, переводящее систему из $x(0) = 0$ в $x(t_1) = x_1$ за время t_1 . Выполнить моделирование системы, демонстрирующее корректность выполненных расчетов.
-

1.2 Ход работы

1) Исследование управляемости системы

а) Матрица управляемости

А можно ли вообще управлять нашей системой? Чтобы узнать это, построим специальную таблицу чисел (матрицу управляемости), которая показывает связь между нашими командами и состоянием объекта.

Для лучшего понимания сути проверки можно провести аналогию: это как проверить, реагирует ли компьютерная мышь на движения и клики, прежде чем идти в

рейтинговый матч по Доте. Если курсор не двигается или герой не слушается – играть невозможно, тебя просто уничтожат на лайне.

Так и в нашей задаче: прежде чем искать управляющее воздействие, переводящее систему из одного состояния в другое, нужно убедиться, что система вообще "слушается" входных сигналов.

Матрица управляемости определяется по формуле:

$$U = \begin{bmatrix} B & AB & A^2B \end{bmatrix}$$

Для нашей задачи она примет вид:

$$U = \begin{bmatrix} -2 & 8 & -26 \\ 4 & -12 & 16 \\ -2 & 10 & -14 \end{bmatrix}$$

Ее ранг равен:

$$\text{rank}(U) = 3$$

Наша система состоит из трех частей (трех переменных). Ранг тоже равен трем. Когда эти числа совпадают – это отличный результат. Это значит: система полностью управляема.

Грубо говоря, у нас есть "контроль" над всеми процессами внутри. Нет таких уголков системы, куда наш сигнал управления не может добраться. Мы можем влиять на нее целиком.

Такую проверку делают инженеры, когда создают любые автоматические системы: автопилот для дрона, управление роботом на заводе, поддержание температуры в печи и так далее.

Сначала нужно убедиться: а можно ли вообще управлять этим объектом? Если проверка прошла успешно (как у нас сейчас) – значит, можно настраивать систему. Если бы проверка не прошла – значит, проблема в самой конструкции объекта, и нужно менять "железо", а не программу.

Результат меня порадовал: система "здоровая" и полностью подконтрольна.

Примечание. Все вычисления выполнены в среде MATLAB с использованием встроенных функций `rank`, `size`, `ctrb`. Пример листинга представлен в приложении [A](#) (листинг [A.1](#))

б) Собственные числа и матрицы Хаутуса

Теперь проверим управляемость другим способом – через собственные числа матрицы A . Это как посмотреть на систему "изнутри": какие у нее есть внутренние режимы (собственные числа) и все ли они поддаются управлению.

Собственные числа – это числа, которые показывают, как система ведет себя сама по себе (без управления). Они находятся из уравнения:

$$\det(\lambda I - A) = 0$$

Для нашей матрицы A собственные числа равны:

$$\lambda_1 = -1$$

$$\lambda_2 = -2 + 3i$$

$$\lambda_3 = -2 - 3i$$

Собственные числа показывают, как система ведёт себя сама по себе, без управления. Отрицательные вещественные части $(-1, -2)$ означают, что система устойчива – со временем затухает. Комплексные числа $(-2 \pm 3i)$ говорят о наличии колебаний.

Матрица Хаутуса для каждого собственного числа λ_i строится по формуле:

$$H_i = \begin{bmatrix} \lambda_i I - A & B \end{bmatrix}$$

Эта матрица показывает: "достаточно ли хорошо управление B влияет на режим с собственным числом λ_i ?"

1. Для $\lambda_1 = -1$:

$$H_1 = \begin{bmatrix} -2 & -2 & 1 & -2 \\ 6 & 6 & 2 & 4 \\ -6 & -6 & -2 & -2 \end{bmatrix}$$

$$\text{rank}(H_1) = 3$$

2. Для $\lambda_2 = -2 + 3i$:

$$H_2 = \begin{bmatrix} -3 + 3i & -2 & 1 & -2 \\ 6 & 5 + 3i & 2 & 4 \\ -6 & -6 & -3 + 3i & -2 \end{bmatrix}$$

$$\text{rank}(H_2) = 3$$

3. Для $\lambda_3 = -2 - 3i$:

$$H_3 = \begin{bmatrix} -3 - 3i & -2 & 1 & -2 \\ 6 & 5 - 3i & 2 & 4 \\ -6 & -6 & -3 - 3i & -2 \end{bmatrix}$$

$$\text{rank}(H_3) = 3$$

Если ранг матрицы Хаутуса равен n (порядку системы, то есть 3), значит соответствующее собственное число управляемо. Все три ранга равны 3 – это замечательно!

Значит, все собственные числа (все внутренние режимы системы) поддаются управлению.

Этот метод (через собственные числа и матрицу Хаутуса) нужен, чтобы понять, какие именно внутренние режимы системы не слушаются управления.

Например, когда создают автопилот для дрона или самолета важно проверить: все ли вибрации и колебания можно контролировать? Если какая-то вибрация окажется неуправляемой – дрон может начать трясти так, что он развалится, и никакое управление не поможет.

Меня порадовало, что все три собственных числа управляемы. Это как проверить каждую кнопку на мышке по отдельности: левая работает, правая работает, колесико тоже. Все в порядке!

Теперь я спокойна: в нашей системе нет "скрытых" режимов, которые могли бы неожиданно выйти из-под контроля.

Важно, что этот метод подтвердил мой предыдущий вывод (через матрицу управляемости). Я точно не ошиблась: система действительно полностью управляема.

Примечание. Все вычисления выполнены в среде MATLAB с использованием встроенных функций `eig`, `size`, `rank`. Пример листинга представлен в приложении [A](#) (листинг [A.2](#))

в) Вещественная Жорданова форма

А можно ли увидеть внутреннюю структуру системы "в чистом виде"? Чтобы узнать это, приведем матрицу A к вещественной Жордановой форме. Это как разобрать сложный механизм на отдельные детали, чтобы проверить каждую по отдельности.

Для лучшего понимания сути метода можно провести аналогию: это как разложить музыку на отдельные инструменты – бас, барабаны, вокал – чтобы проверить, слышен ли каждый из них. Если какой-то инструмент заглушен другими, микс неудачный.

Так и в нашей задаче: прежде чем делать выводы об управляемости, нужно "разложить" систему по собственным числам и проверить, влияет ли управление на каждое из них.

Из предыдущего пункта знаю, что собственные числа:

$$\lambda_1 = -1$$

$$\lambda_2 = -2 + 3i$$

$$\lambda_3 = -2 - 3i$$

Собственный вектор находится из уравнения:

$$(A - \lambda I)v = 0$$

Для комплексного собственного числа $\lambda_2 = -2 + 3i$ получаю комплексный собственный вектор v_2 , который разделяю на вещественную и мнимую части:

$$v_2 = \operatorname{Re}(v_2) + i \cdot \operatorname{Im}(v_2) = \begin{bmatrix} -0.3162 \\ 0.6325 \\ -0.6325 \end{bmatrix} + i \begin{bmatrix} -0.3162 \\ 0 \\ 0 \end{bmatrix}$$

Для вещественного собственного числа $\lambda_1 = -1$ собственный вектор:

$$v_1 = \begin{bmatrix} -0.7071 \\ 0.7071 \\ 0 \end{bmatrix}$$

Каждый собственный вектор – это особое направление в системе. Если система начнет двигаться вдоль этого направления, она так и продолжит, не отклоняясь. Это как главные оси у эллипса.

Матрица перехода строится из вещественной части, мнимой части комплексного вектора и вещественного собственного вектора:

$$P = [\operatorname{Re}(v_2) \quad \operatorname{Im}(v_2) \quad v_1] = \begin{bmatrix} -0.3162 & -0.3162 & -0.7071 \\ 0.6325 & 0 & 0.7071 \\ -0.6325 & 0 & 0 \end{bmatrix}$$

Нахожу обратную матрицу:

$$P^{-1} = \begin{bmatrix} 0 & 0 & -1.5811 \\ -3.1623 & -3.1623 & -1.5811 \\ 0 & 1.4142 & 1.4142 \end{bmatrix}$$

Матрица P – это "мост" между старой и новой системами координат. В новых координатах система выглядит проще – видно, из каких независимых частей она состоит.

Теперь вычисляю вещественную Жорданову форму по формуле $J_R = P^{-1}AP$:

$$J_R = \begin{bmatrix} -2 & 3 & 0 \\ -3 & -2 & 0 \\ 0 & 0 & -1 \end{bmatrix} = \begin{bmatrix} J_1 & 0 \\ 0 & J_2 \end{bmatrix}$$

где Жордановы блоки:

$$J_1 = \begin{bmatrix} -2 & 3 \\ -3 & -2 \end{bmatrix}, \quad J_2 = [-1]$$

Система распалась на две независимые части: колебательное звено (блок 2×2) и простое апериодическое звено (элемент -1). Теперь каждую часть можно анализировать отдельно, что гораздо проще, чем работать с исходной запутанной системой.

Выполняю замену координат:

$$x = P\tilde{x}, \quad \text{откуда} \quad \tilde{x} = P^{-1}x$$

Тогда система $\dot{x} = Ax + Bu$ в Жордановой форме примет вид:

$$\dot{\tilde{x}} = J_R\tilde{x} + \tilde{B}u$$

Нахожу матрицу управления в новых координатах:

$$\tilde{B} = P^{-1}B$$

После вычисления получаю:

$$\tilde{B} = \begin{bmatrix} 3.1623 \\ -3.1623 \\ 2.8284 \end{bmatrix}$$

Матрица $\tilde{B}$ показывает, как управляющее воздействие u влияет на каждую часть системы в новых координатах.

Если элемент $\tilde{B}$ равен нулю – значит, управление не доходит до соответствующей моды (части системы). Эта часть будет "неуправляема".

В нашем случае все элементы ненулевые – управление влияет на все части системы.

Теперь система в Жордановой форме выглядит так:

$$\dot{\tilde{x}} = \begin{bmatrix} -2 & 3 & 0 \\ -3 & -2 & 0 \\ 0 & 0 & -1 \end{bmatrix} \tilde{x} + \begin{bmatrix} 3.1623 \\ -3.1623 \\ 2.8284 \end{bmatrix} u$$

Первые две строки матрицы $\tilde{B}$ (3.1623 и -3.1623) относятся к колебательному блоку J_1 . Они ненулевые – колебательная часть системы управляема.

Третья строка (2.8284) относится к блоку J_2 (затухание). Она тоже ненулевая – эта часть тоже управляема.

Итак, все части системы управляемы, система полностью управляема.

Такое разделение помогает понять, как система будет вести себя на практике: будут ли колебания, как быстро они затухнут, и можно ли всем этим управлять. В нашем случае все элементы управляемы – значит, мы можем подавить любые колебания системы через управление. Это важно, например, для автопилота (чтобы гасить качку самолета) или стабилизации робота.

Мне понравилось, как Жорданова форма "прояснила" картину. Если раньше матрица A выглядела как набор непонятных чисел, то теперь я четко вижу: система состоит из колебательной части и простой затухающей. Это как надеть очки и увидеть детали, которые раньше были размыты.

Результат меня порадовал: все три строки $\tilde{B}$ ненулевые, значит управление "достает" до каждого режима.

Примечание. Все вычисления выполнены в среде MATLAB с использованием встроенных функций `eig`, `inv`. Пример листинга представлен в приложении [A](#) (листинг [A.3](#))

2) Грамиан управляемости

На предыдущем этапе я проверила, управляема система или нет. Теперь мне нужно оценить, насколько легко ей управлять. Для этого вычисляю Грамиан управляемости P на интервале $t_1 = 3$.

В отличие от ранга матрицы управляемости, Грамиан дает количественную оценку: он показывает, сколько энергии потребуется, чтобы перевести систему из нулевого состояния в требуемое. Если собственные числа Грамиана большие и положительные – управление "дешевое". Если близки к нулю – системе потребуется огромная мощность приводов.

Теоретически Грамиан задаётся интегралом:

$$P(t) = \int_0^{t_1} e^{At} B B^T e^{A^T t} dt,$$

Пример вычисления Грамина в среде MATLAB представлен в приложении [A](#) (листинг [A.4](#))

После запуска скрипта получила матрицу Грамиана для $t_1 = 3$:

$$P(3) = \begin{bmatrix} 1.0078 & -1.4438 & 0.1155 \\ -1.4438 & 2.5336 & -0.5385 \\ 0.1155 & -0.5385 & 0.5385 \end{bmatrix}$$

Ее собственные числа:

$$\lambda_1 = 3.4957$$

$$\lambda_2 = 0.0575$$

$$\lambda_3 = 0.5266$$

Анализ результатов.

1. Все собственные числа строго положительны. Это математически подтверждает вывод из пункта 1: система полностью управляема. Если бы хотя бы одно число было ≤ 0 , существовало бы направление в пространстве состояний, куда систему невозможно перевести конечным управлением.
2. Минимальное собственное число $\lambda_{\min} = 0.0575$ показывает "самое слабое" направление управления. Чем оно дальше от нуля, тем меньше энергии нужно для разгона системы в этом направлении.

3) Управление для перевода системы

Теперь мне нужно найти конкретное управляющее воздействие $u(t)$, которое переведет систему из начального состояния $x(0) = 0$ в требуемое конечное состояние $x(t_1) = x_1$ за время $t_1 = 3$ секунды.

На предыдущих этапах я проверила, что система управляема (теоретически). Теперь нужно построить реальное управление – конкретный сигнал $u(t)$, который фактически переведет систему в нужную точку.

Это как перейти от вопроса "можно ли доехать из Казани в Петербург?" к вопросу "какой маршрут выбрать и сколько бензина понадобится?".

Управление, переводящее систему из $x(0) = 0$ в $x(t_1) = x_1$ за время $t_1 = 3$, находится по формуле:

$$u(t) = B^T e^{A^T(t_1-t)} (P(t_1))^{-1} x_1$$

Вычисляю управление $u(t)$ в MATLAB на интервале от 0 до 3 секунд.

Затем с помощью `ode45` решаю систему дифференциальных уравнений – это покажет траекторию движения системы под действием найденного управления.

Полученное управление имеет вид:

$$u(t) = \begin{bmatrix} 2 \\ -3 \\ 3 \end{bmatrix}$$

На рисунке 1 показано управляющее воздействие $u(t)$ во времени. Видно, что управление начинается с нулевого значения, затем плавно растет, достигая максимума около 2.6 секунд (значение почти 4), после чего резко падает. Такое поведение характерно для систем, которые сначала "разгоняются", а потом "тормозят" для достижения цели.

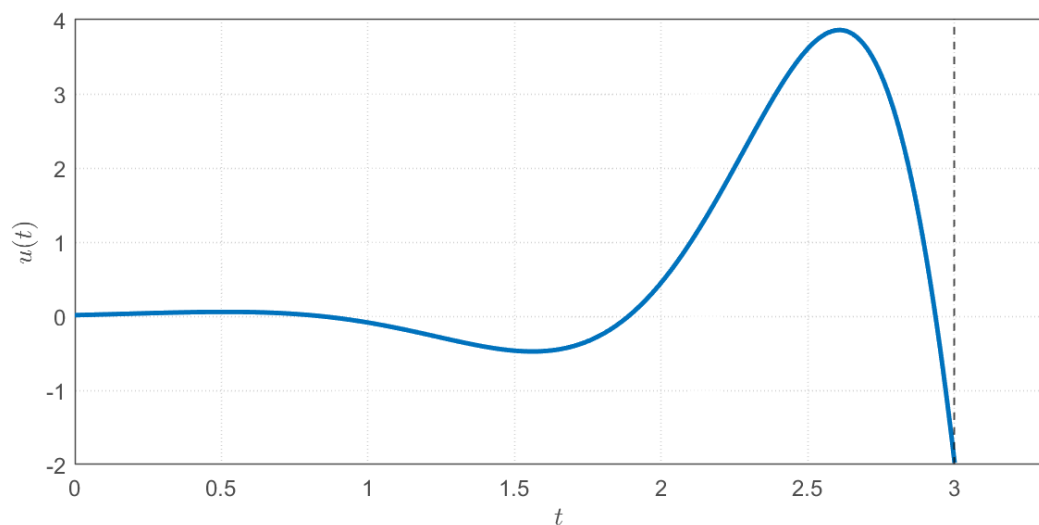


Рисунок 1 — График управляющего воздействия $U(t)$

На рисунке 2 показаны траектории всех трех координат системы $x_1(t)$, $x_2(t)$, $x_3(t)$. Все координаты начинаются из нуля (начальное условие $x(0) = 0$) и под действием найденного управления приходят к требуемым конечным значениям к моменту $t = 3$ секунды. Видно, что координаты ведут себя по-разному: x_1 и x_3 плавно растут до положительных значений (2 и 3 соответственно), а x_2 уходит в отрицательную область (-3). Это подтверждает, что управление работает корректно – система переходит из начального состояния в конечное.

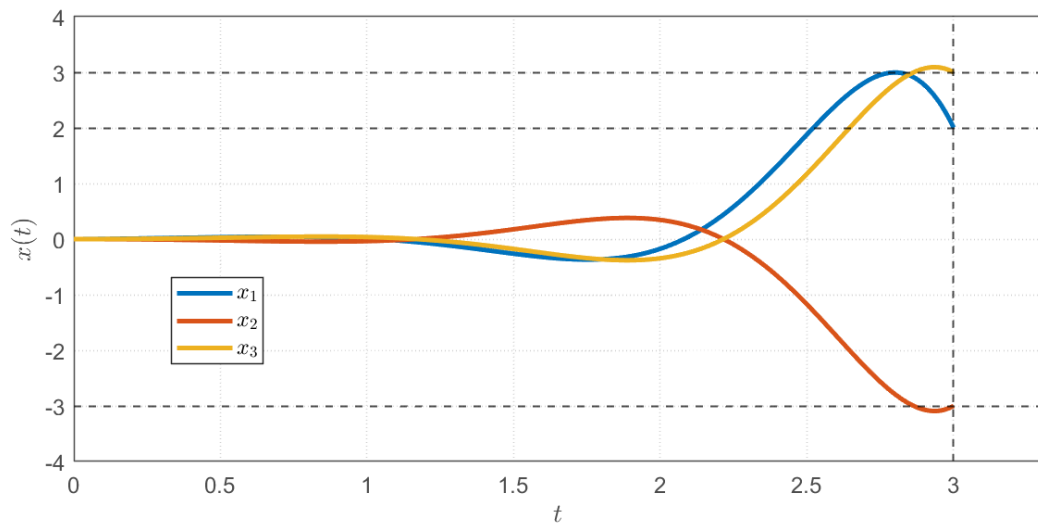


Рисунок 2 — Графики компонент вектора состояния $x(t)$

Такой подход (нахождение управления через Грамиан) используется в реальных задачах. Например, в робототехнике – для расчета управляющих сигналов моторам манипулятора, чтобы он переместился из точки А в точку В за заданное время.

Я поняла, что главное преимущество управления через Грамиан в том, что оно минимизирует энергию. Это критично, когда ресурсы ограничены – например, батарея дрона или топливо ракеты.

Мне понравилось, что формула сработала точно. Когда я увидела на графике, что координаты действительно приходят к требуемым значениям, стало понятно: вся предыдущая работа (проверка управляемости, вычисление Грамиана) была не зря.

Особенно интересно было увидеть форму графика $u(t)$. Оказалось, что управление меняется неравномерно: сначала оно почти нулевое, потом небольшое отрицательное, затем растет до максимума около 2.6 секунд, и в конце резко падает. Это логично: сначала система "готовится", потом активно действует, и в конце тормозит у цели.

Ну, а с примером вычисления управления и построения графиков можно ознакомиться в приложении А (листинг А.5)

1.3 Выводы

1. Система полностью управляема, что подтверждается:
 - Рангом матрицы управляемости ($\text{rank } U = 3$)
 - Рангами матриц Хаутуса для всех собственных чисел
 - Структурой жордановой формы.
2. Грамиан управляемости положительно определен, все его собственные числа положительны.
3. Найдено управление, которое переводит систему из начального состояния в заданное конечное состояние за время $t_1 = 3$.
4. Результаты моделирования подтверждают корректность выполненных расчетов.

2 Еще одно исследование управляемости

2.1 Задание

Система описывается уравнением состояния:

$$\dot{x} = Ax + Bu$$

где:

$$A = \begin{bmatrix} 1 & 2 & -1 \\ -6 & -7 & -2 \\ 6 & 6 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}, \quad x'_1 = \begin{bmatrix} 2 \\ -3 \\ 3 \end{bmatrix}, \quad x''_1 = \begin{bmatrix} 1 \\ -2 \\ 3 \end{bmatrix}$$

Необходимо:

1. Проверить обе точки x'_1 и x''_1 на принадлежность управляемому подпространству системы. Принять целевой точкой x_1 ту, что принадлежит.
2. Выполнить все шаги Задания 1 для рассматриваемой системы и выбранной целевой точки x_1 :
 - Исследовать управляемость системы (матрица управляемости, собственные числа, Жорданова форма)
 - Найти Грамиан управляемости для времени $t_1 = 3$
 - Найти управление, переводящее систему из $x(0) = 0$ в $x(t_1) = x_1$
 - Выполнить моделирование системы.

2.2 Ход работы

1) Прежде чем искать управление, которое переведет систему в нужную точку, я должна убедиться, что эта точка вообще достижима. Это как планировать поездку: сначала нужно проверить, есть ли дорога до места назначения, иначе сколько ни ездить – не доедешь. Так и здесь: если целевая точка лежит вне возможностей системы, никакое управление ее туда не переведет.

Матрица управляемости U задает "зону досягаемости" системы. Чтобы проверить, попадает ли в нее точка x_1 , сравниваю два ранга:

$$\text{rank}(U) \quad \text{и} \quad \text{rank}[U \quad x_1]$$

Если ранги равны – точку можно достичь конечным управлением. Если ранг расширенной матрицы больше – точка лежит "вне" возможностей системы.

Сначала я построила матрицу управляемости U для новой матрицы $B = [2; 0; 0]^T$, чтобы понять, насколько вообще управляема эта система и какие точки она может достичь.

$$U = \begin{bmatrix} B & AB & A^2B \end{bmatrix} = \begin{bmatrix} 2 & 2 & -34 \\ 0 & -12 & 48 \\ 0 & 12 & -48 \end{bmatrix}$$

Ранг этой матрицы оказался равен 2 (а не 3, как было в первом задании).

$$\text{rank}(U) = 2$$

Это важный результат: система не полностью управляема. Она может двигаться только в двумерном подпространстве (по плоскости), а не в любой точке трёхмерного пространства. Это происходит потому, что вторая и третья строки матрицы U противоположны $(0, -12, 48)$ и $(0, 12, -48)$, то есть зависимы.

Теперь проверяю, лежат ли целевые точки в этом плоском подпространстве.

1. Для точки $x'_1 = [2; -3; 3]^T$:

$$\text{rank}([U \quad x'_1]) = 2$$

Ранг расширенной матрицы $[U \quad x_1]$ тоже равен 2.

Ранги совпадают – точка принадлежит управляемому подпространству. В нее можно попасть.

2. Для точки $x''_1 = [1; -2; 3]^T$:

$$\text{rank}([U \quad x''_1]) = 3$$

Ранг расширенной матрицы $[U \quad x''_1]$ стал 3.

Ранг увеличился – точка НЕ принадлежит подпространству. Система физически не может попасть в эту точку при текущем расположении матрицы управляемости B .

Итак, я выбираю в качестве целевой точки $x_1 = x'_1 = [2; -3; 3]^T$, так как только она достижима.

Такая ситуация (когда система не полностью управляема) часто встречается на практике:

Робот-манипулятор застрял: Если один из моторов сломался (аналог нашей матрицы B), робот теряет степень свободы. Он может двигать руку только в одной плоскости. Если оператор попросит взять деталь "сверху" (точка x''_1), робот не сможет. А если "сбоку" (точка x'_1) – сможет.

Дрон с отказавшим пропеллером: Если один мотор отказал, дрон не может подниматься строго вертикально, но все еще может маневрировать в горизонтальной плоскости.

Меня удивило, что ранг матрицы упал до 2. В первом задании система была полностью управляема, а изменение матрицы B "сломало" управляемость по одному из направлений.

Это наглядно показало, насколько важно, куда приложено управление. Точка x''_1 оказалась для системы "недостижимой мечтой" – она существует, но попасть туда при такой структуре управления невозможно. Поэтому выбор точки x'_1 был единственно верным решением.

Скрипт, реализующий проверку принадлежности целевых точек управляемому подпространству линейной системы, приведен в приложении Б (листинг Б.1)

Собственные числа матрицы A известны из предыдущего задания, проверим управляемость системы с помощью матрицы Хаутуса.

1. Для $\lambda_1 = -2 + 3i$:

$$H_1 = \begin{bmatrix} -3 + 3i & -2 & 1 & 2 \\ 6 & 5 + 3i & 2 & 0 \\ -6 & -6 & -3 + 3i & 0 \end{bmatrix}$$

$$\text{rank}(H_1) = 3$$

2. Для $\lambda_2 = -2 - 3i$:

$$H_2 = \begin{bmatrix} -3 - 3i & -2 & 1 & 2 \\ 6 & 5 - 3i & 2 & 0 \\ -6 & -6 & -3 - 3i & 0 \end{bmatrix}$$

$$\text{rank}(H_2) = 3$$

3. Для $\lambda_3 = -1$:

$$H_3 = \begin{bmatrix} -2 & -2 & 1 & 2 \\ 6 & 6 & 2 & 0 \\ -6 & -6 & -2 & 0 \end{bmatrix}$$

$$\text{rank}(H_3) = 2$$

Первые два собственных числа (комплексные, $\lambda_{1,2} = -2 \pm 3i$) управляемы – их матрицы Хаутуса имеют полный ранг 3.

А вот третье собственное число ($\lambda_3 = -1$) неуправляемо – ранг матрицы H_3 равен 2, а не 3. Это означает, что соответствующий режим системы не поддается управлению при текущей матрицы B .

Это полностью согласуется с результатом проверки через матрицу управляемости: $\text{rank}(U) = 2$, то есть система имеет только 2 управляемых направления из 3.

Код программы, реализующий проверку управляемости по критерию Хаутуса, приведен в Приложении Б (листинг Б.2)

Теперь приведу систему к вещественной Жордановой форме, чтобы наглядно увидеть структуру системы и понять, какие режимы управляемы.

Собственные векторы, матрица перехода P и вещественная жорданова форма J зависят только от матрицы A . Поскольку она совпадает с матрицей из задания 1, данные характеристики остались прежними.

Но вектор управления в новом базисе изменился:

$$\tilde{B} = P^{-1}B = \begin{bmatrix} -0.0000 \\ -6.3246 \\ 0.0000 \end{bmatrix}$$

В новых координатах Жорданова форма запишется следующим образом:

$$\dot{\tilde{x}} = \begin{bmatrix} -2 & 3 & 0 \\ -3 & -2 & 0 \\ 0 & 0 & -1 \end{bmatrix} \tilde{x} + \begin{bmatrix} -0.0000 \\ -6.3246 \\ 0.0000 \end{bmatrix} u$$

Итак:

1. Блок J_1 (колебательный режим $\lambda_{1,2} = -2 \pm 3i$): из первых двух строк $\tilde{B}$ только вторая ненулевая (-6.3246), первая равна нулю (-0.0000). Поскольку хотя бы одна строка ненулевая, колебательная часть системы управляема.
2. Блок J_2 (простой режим $\lambda_3 = -1$): третья строка $\tilde{B}$ равна нулю. Этот режим неуправляем.

При такой структуре матрицы B система может двигаться только в двумерной плоскости. Третье направление полностью отключено от управления. Никаким сигналом

$u(t)$ мы не сможем повлиять на эту компоненту состояния.

Такая ситуация типична при частичном отказе исполнительных механизмов. Например:

У дрона отказал один мотор. Он все еще может парить и поворачивать, но теряет способность компенсировать определенные типы колебаний.

В станке с ЧПУ сломался привод по одной оси. Деталь можно обработать только в оставшейся плоскости.

Меня удивило, как одна измененная строка в B кардинально меняет картину. Раньше система была полностью управляемой, а теперь "потеряла" одно направление. Это наглядно показало, что управляемость – это свойство не самой матрицы A , а пары (A, B) .

Жорданова форма помогла точно локализовать "сломанный" режим. Теперь я понимаю, почему в предыдущем пункте было так важно проверять принадлежность целевой точки управляемому подпространству: если бы я выбрала точку, имеющую компоненту по неуправляемому направлению, система физически не смогла бы туда попасть, сколько бы энергии мы ни тратили.

Код для нахождения вещественной жордановой формы представлен в приложении Б (листинг Б.3)

Вычислим Грамиан управляемости P для времени $t_1 = 3$ секунды. В предыдущих пунктах я поняла, что система управляема не полностью ($\text{rank} = 2$). Грамиан позволит увидеть это в цифрах и оценить, какие именно направления движения доступны, а какие – нет.

Полученная матрица Грамиана:

$$P(3) = \begin{bmatrix} 1.4615 & -1.1538 & 1.1538 \\ -1.1538 & 1.3846 & -1.3846 \\ 1.1538 & -1.3846 & 1.3846 \end{bmatrix}$$

Ее собственные числа:

$$\lambda_1 = 3.8733$$

$$\lambda_2 = 0.3575$$

$$\lambda_3 = 0$$

1. Два положительных числа (3.87 и 0.36) означают, что система управляема по двум направлениям. Мы можем переводить ее в любые точки, лежащие в этой двумерной плоскости.
2. Третье число равно нулю. Это вырожденный Грамиан. Математически это значит, что существует одно направление в пространстве состояний, куда система не может попасть никаким управлением $u(t)$.

Это полностью подтверждает мой вывод из пункта про матрицу Хаутуса и Жорданову форму: режим с собственным числом $\lambda = -1$ оказался неуправляемым.

Меня поразило, как точно математика описывает физику. Нулевое собственное число – это не просто абстракция, а реальная "мертвая зона" системы.

Раньше, когда система была полностью управляемой, все числа были положительными. Теперь одно из них исчезло. Это наглядно показывает: изменив матрицу B , мы буквально "отрезали" системе возможность двигаться в одном из измерений. Теперь я точно знаю, что целевую точку нужно выбирать только в той плоскости, где Грамиан ненулевой.

Код нахождения грамиана представлен в приложении [Б](#) (листинг [Б.4](#))

Теперь нам нужно найти конкретное управляющее воздействие $u(t)$, которое переведет систему из начального состояния $x(0) = 0$ в выбранную целевую точку $x(t_1) = x_1 = [2; -3; 3]$ за время $t_1 = 3$ секунды.

Важный момент: Поскольку система не полностью управляема (Грамиан вырожденный, $\text{rank} = 2$), я не могу использовать обычную обратную матрицу $P(t_1)^{-1}$ – она не существует. Вместо этого использую псевдообратную матрицу $(P(t_1))^+$, которая вычисляется командой MATLAB `pinv(P)`.

Использую формулу:

$$u(t) = B^T e^{A^T(t_1-t)} (P(t_1))^+ x_1$$

Псевдообратная матрица автоматически "игнорирует" неуправляемые направления и находит управление, которое переводит систему в целевую точку только по управляемым направлениям.

После нахождения $u(t)$ решаю систему дифференциальных уравнений численно (команда MATLAB `ode45`), чтобы проверить, действительно ли система приходит в точку x_1 .

На рисунке [3](#) показано управляющее воздействие $u(t)$ во времени. Видно, что управление начинается с нулевого значения, затем уходит в отрицательную область (достигая примерно -4 к 1.5 секунде), после чего активно растёт до максимума около 7 к 2.6 секунде, и в конце резко падает до -15 .

По сравнению с предыдущим случаем амплитуды управления значительно выросли – это связано с тем, что система теперь частично неуправляема, и для достижения цели требуется больше энергии. Такое поведение характерно для систем с ограниченной управляемостью: управление вынуждено "компенсировать" недостающие степени свободы, работая на пределе возможностей.

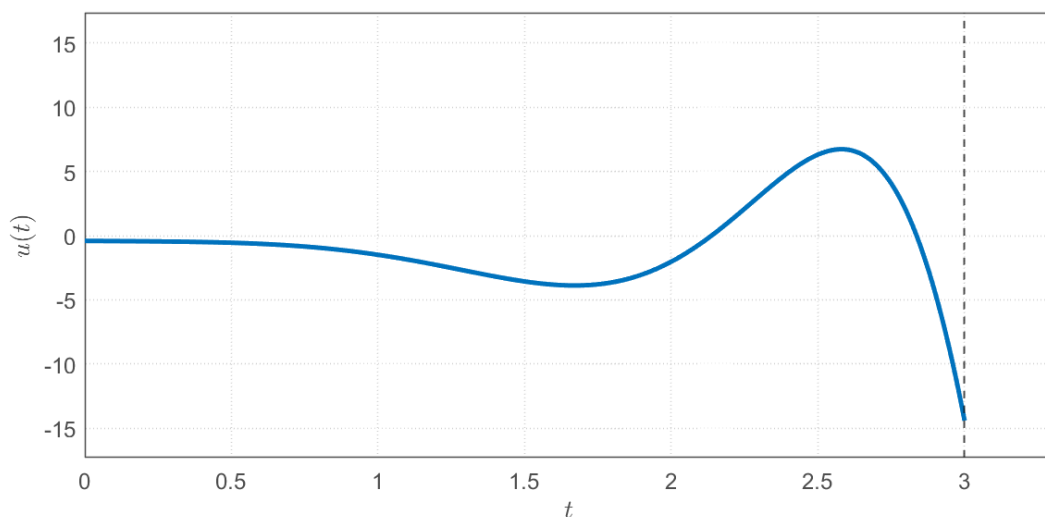


Рисунок 3 — График управляющего воздействия $U(t)$

На рисунке 4 показаны траектории всех трех координат системы $x_1(t)$, $x_2(t)$, $x_3(t)$. Все координаты начинаются из нуля (начальное условие $x(0) = 0$) и под действием найденного управления приходят к требуемым конечным значениям к моменту $t = 3$ секунды.

Видно, что координаты ведут себя более "нервно", чем в полностью управляемом случае:

- x_1 (синяя) колеблется вокруг нуля, прежде чем достичь значения 2;
- x_2 (красная) уходит в глубокий минус (до -4), затем резко растет до $+4$, и только в конце падает до -3 ;
- x_3 (желтая) медленно набирает значение, проходя через отрицательную область, прежде чем достичь 3.

Несмотря на сложные траектории, все координаты приходят точно в нужные точки. Это подтверждает, что управление работает корректно, хотя и требует больших затрат энергии из-за частичной неуправляемости системы.

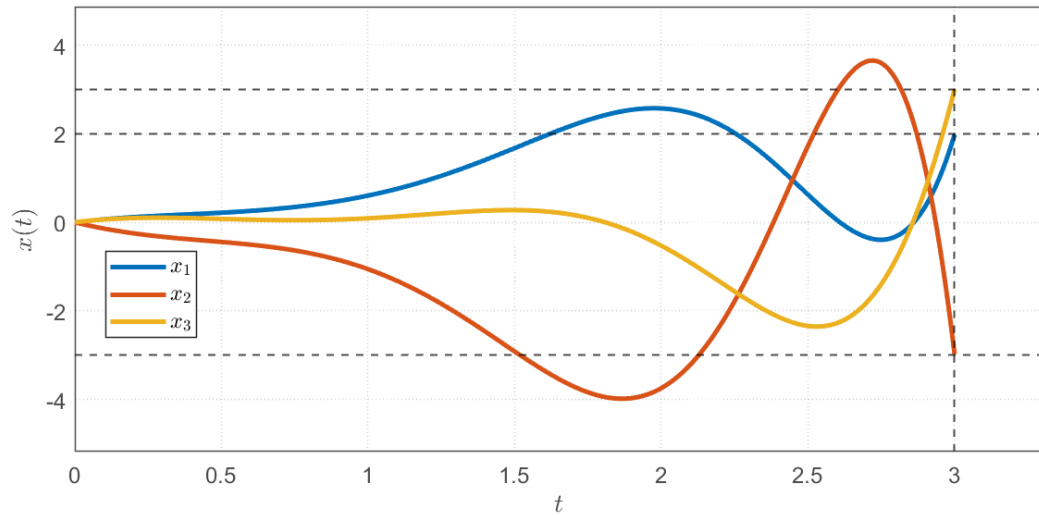


Рисунок 4 — Графики компонент вектора состояния $x(t)$

Код для вычисления управления и построения графиков приведен в приложении Б (листинг Б.5)

2.3 Выводы

1. Из двух проверяемых точек управляемому подпространству принадлежит только $x'_1 = [2; -3; 3]^T$.
2. Система не полностью управляема ($\text{rank } U = 2 < 3$).
3. Грамиан управляемости вырожден, что подтверждает неполную управляемость.
4. Для выбранной точки найдено управление
5. Результаты моделирования подтверждают теоретические выводы.

3 Исследование наблюдаемости

3.1 Задание

Система описывается системой уравнений:

$$\begin{cases} \dot{x} = Ax \\ y = Cx \end{cases}$$

где:

$$A = \begin{bmatrix} -9 & 0 & -10 \\ -4 & -1 & -6 \\ 6 & -2 & 5 \end{bmatrix}, \quad C = \begin{bmatrix} 2 & -1 & 2 \end{bmatrix}, \quad f(t) = -3e^{-3t} \cos(2t) - 2e^{-3t} \sin(2t)$$

Необходимо:

1. Исследовать наблюдаемость системы:

- а) Найти матрицу наблюдаемости системы, определить её ранг, сделать вывод об наблюдаемости системы в целом.
 - б) Найти собственные числа матрицы A , найти для каждого из собственных чисел матрицу Хаутуса (для наблюдаемости), определить ранги матриц, сделать выводы об наблюдаемости каждого собственного числа и системы в целом.
 - в) Найти вещественную Жорданову форму системы и сделать выводы об наблюдаемости каждого собственного числа и системы в целом.
2. Найти Грамиан наблюдаемости системы относительно времени $t_1 = 3$, вычислить его собственные числа. Проанализировать полученные собственные числа с точки зрения наблюдаемости системы.
3. Считая, что выход системы $y(t)$ подчиняется закону $y(t) = f(t)$ на временном интервале $t \in [0, t_1]$, определить начальные условия системы. Выполнить моделирование системы, демонстрирующее корректность выполненных расчётов.
-

3.2 Ход работы

Прежде чем восстанавливать начальное состояние системы по измерениям выхода $y(t)$, нужно убедиться, что это вообще возможно. Для этого построю матрицу наблюдаемости V – она показывает, можно ли по выходному сигналу $y(t)$ однозначно определить все внутренние переменные состояния x_1, x_2, x_3 .

Это как попытаться понять, что происходит внутри черного ящика, глядя только на его выход. Если матрица наблюдаемости имеет полный ранг – мы "видим" всю систему. Если нет – какие-то процессы останутся скрытыми.

Матрица наблюдаемости определяется по формуле:

$$V = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix}$$

В нашем случае матрица A примет вид:

$$V = \begin{bmatrix} 2 & -1 & 2 \\ -2 & -3 & -4 \\ 6 & 11 & 18 \end{bmatrix}$$

Ранг матрицы наблюдаемости:

$$\text{rank}(V) = 3$$

Поскольку ранг матрицы наблюдаемости равен 3, система полностью наблюдаема. Это значит, что по измерениям выхода $y(t)$ можно восстановить все три переменные состояния x_1, x_2, x_3 . Никакая внутренняя динамика не ускользнет от наблюдения.

Проверка наблюдаемости критически важна на практике.

Например, в диагностике оборудования. По вибрациям можно определить износ всех деталей. Или в навигации: по показаниям гироскопов и акселерометров точно определить положение объекта

Если бы система была ненаблюдаема, инженер понял бы: нужно добавлять датчики или менять их расположение.

Результат меня порадовал: ранг матрицы наблюдаемости равен 3, значит система полностью наблюдаема. Наш датчик C выбран удачно, и мы можем восстановить полное состояние системы. Это дает уверенность, что следующую задачу (восстановление начального состояния) мы решим правильно.

Код для нахождения матрица наблюдаемости представлен в приложении [В](#) (листинг [В.1](#))

Теперь проверим наблюдаемость более детально – для каждого собственного числа отдельно. Матрица наблюдаемости показала общую картину, а матрицы Хаутуса помогут увидеть, нет ли отдельных "невидимых" режимов системы.

Матрица Хаутуса для наблюдаемости определяется по формуле:

$$H_i = \begin{bmatrix} \lambda_i I - A \\ C \end{bmatrix}$$

Собственные числа матрицы A :

$$\lambda_1 = 1$$

$$\lambda_2 = -3 + 2i$$

$$\lambda_3 = -3 - 2i$$

Построим матрицу Хаутуса для каждого собственного числа:

1. Для $\lambda_1 = 1$:

$$H_1 = \begin{bmatrix} 10 & 0 & 10 \\ 4 & 2 & 6 \\ -6 & 2 & -4 \\ 2 & -1 & 2 \end{bmatrix}$$

$$\text{rank}(H_1) = 3$$

2. Для $\lambda_2 = -3 + 2i$:

$$H_2 = \begin{bmatrix} 6 + 2i & 0 & 10 \\ 4 & -2 + 2i & 6 \\ -6 & 2 & -8 + 2i \\ 2 & -1 & 2 \end{bmatrix}$$

$$\text{rank}(H_2) = 3$$

3. Для $\lambda_3 = -3 - 2i$:

$$H_3 = \begin{bmatrix} 6 - 2i & 0 & 10 \\ 4 & -2 - 2i & 6 \\ -6 & 2 & -8 - 2i \\ 2 & -1 & 2 \end{bmatrix}$$

$$\text{rank}(H_3) = 3$$

Все три матрицы Хаутуса имеют полный ранг 3. Ранг матрицы A тоже имеет ранг 3, значит, система полностью наблюдаема – все ее внутренние режимы "видны" через нашу матрицу C .

Такая проверка важна, когда нужно понять, какие именно колебания или режимы системы мы не видим. Например, вибрации самолета: все ли типы колебаний фиксируют датчики? В химическом реакторе: контролируем ли все опасные режимы?

Если обнаружится ненаблюдаемое собственное число, инженер поймет: именно этот режим может стать "неожиданностью" – система может войти в него, а мы даже не заметим.

Мне понравилось, как матрицы Хаутуса "разложили" наблюдаемость по полочкам. Теперь я знаю не просто "наблюдаема система или нет" а какие именно режимы видны.

Особенно интересно было работать с комплексными собственными числами $(-3 \pm 2i)$. Они соответствуют колебательным режимам системы. Хорошо, что оба они наблюдаемы – значит, мы сможем отслеживать все колебания системы.

Результат меня порадовал: все собственные числа наблюдаемы. Это полностью согласуется с выводом из предыдущего пункта: система полностью наблюдаема. Теперь я уверена, что смогу восстановить начальное состояние системы по измерениям выхода $y(t)$.

Код для вычисления матриц Хаутуса представлен в приложении В (листинг В.2)

Следующим шагом приведу систему к вещественной Жордановой форме, чтобы наглядно увидеть внутреннюю структуру системы и проверить наблюдаемость каждого режима отдельно через нашу матрицу C .

Из предыдущего пункта знаю, что собственные числа:

$$\lambda_1 = 1$$

$$\lambda_2 = -3 + 2i$$

$$\lambda_3 = -3 - 2i$$

Для комплексного собственного числа $\lambda_2 = -3 + 2i$ получаю комплексный собственный вектор v_2 , который разделяю на вещественную и мнимую части:

$$v_2 = \operatorname{Re}(v_2) + i \cdot \operatorname{Im}(v_2) = \begin{bmatrix} 0.7906 \\ 0.3162 \\ -0.4743 \end{bmatrix} + i \begin{bmatrix} 0 \\ -0.1581 \\ -0.1581 \end{bmatrix}$$

Матрица перехода строится из вещественной части, мнимой части комплексного вектора и вещественного собственного вектора:

$$P = \begin{bmatrix} \operatorname{Re}(v_2) & \operatorname{Im}(v_2) & v_1 \end{bmatrix} = \begin{bmatrix} 0.7906 & 0 & 0.5774 \\ 0.3162 & -0.1581 & 0.5774 \\ -0.4743 & -0.1581 & -0.5774 \end{bmatrix}$$

Нахожу обратную матрицу:

$$P^{-1} = \begin{bmatrix} 0 & 0 & -1.5811 \\ -3.1623 & -3.1623 & -1.5811 \\ 0 & 1.4142 & 1.4142 \end{bmatrix}$$

Теперь вычисляю вещественную Жорданову форму по формуле $J_R = P^{-1}AP$:

$$J_R = \begin{bmatrix} -3 & 0 & 0 \\ -2 & -3 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} J_1 & 0 \\ 0 & J_2 \end{bmatrix}$$

где Жордановы блоки:

$$J_1 = \begin{bmatrix} -3 & 0 \\ -2 & -3 \end{bmatrix}, \quad J_2 = [1]$$

Выполняю замену координат:

$$x = P\tilde{x}, \quad \text{откуда} \quad \tilde{x} = P^{-1}x$$

Тогда система $\dot{\tilde{x}} = A\tilde{x}$ в Жордановой форме примет вид:

$$\dot{\tilde{x}} = J_R \tilde{x} \quad y = \tilde{C} \tilde{x}$$

Нахожу матрицу выхода в новых координатах:

$$\tilde{C} = CP$$

После вычисления получаю:

$$\tilde{C} = \begin{bmatrix} 0.3162 & -0.1581 & -0.5774 \end{bmatrix}$$

Матрица $\tilde{C}$ показывает, как выход $y(t)$ связан с каждой частью системы в новых координатах.

Если элемент $\tilde{C}$ равен нулю – значит, соответствующая мода (часть системы) не наблюдается на выходе. Эта часть будет "ненаблюдаема".

В нашем случае все элементы ненулевые – выход содержит информацию обо всех частях системы.

Теперь система в Жордановой форме выглядит так:

$$\dot{\tilde{x}} = \begin{bmatrix} -3 & 0 & 0 \\ -2 & -3 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tilde{x}, \quad y = \begin{bmatrix} 0.3162 & -0.1581 & -0.5774 \end{bmatrix} \tilde{x}$$

Первые две компоненты матрицы $\tilde{C}$ (0.3162 и -0.1581) относятся к колебательному блоку J_1 . Они ненулевые – колебательная часть системы наблюдаема.

Третья компонента (-0.5774) относится к блоку J_2 . Она тоже ненулевая – эта часть тоже наблюдаема.

Итак, все части системы наблюдаемы, система полностью наблюдаема.

Это полностью согласуется с предыдущими выводами.

Жорданова форма особенно полезна инженерам. Например, в фильтрация шумов: зная структуру системы, можно отделить полезные сигналы от помех.

Приятно, что все три метода (матрица наблюдаемости, матрицы Хаутуса и Жорданова форма) дали одинаковый результат. Это как три независимых эксперта подтвердили: система полностью наблюдаема.

Код для нахождения Жордановой формы представлени в приложении [В](#) (листинг [В.3](#))

Далее вычислим Грамиан наблюдаемости $Q(t_1)$ для времени $t_1 = 3$ секунды. Этот Грамиан показывает не просто "да/нет" (наблюдаема система или нет), а насколько качественно мы можем восстановить состояние системы по измерениям выхода $y(t)$.

Грубо говоря, Грамиан – это "качество картинки" нашей системы наблюдения. Если все собственные числа большие и положительные – картинка четкая. Если есть очень маленькие числа – какие-то процессы системы будут видны "размыто" или могут потеряться на фоне шума датчиков.

Грамиан наблюдаемости вычисляется по формуле:

$$Q(t_1) = \int_0^{t_1} e^{A^T t} C^T C e^{A t} dt$$

Матрица Грамиана наблюдаемости примет вид:

$$Q(3) = \begin{bmatrix} 201.8539 & -201.7517 & 201.5658 \\ -201.7517 & 201.7392 & -201.4764 \\ 201.5658 & -201.4764 & 201.3034 \end{bmatrix}$$

Ее собственные числа:

$$\lambda_1 = 604.8284$$

$$\lambda_2 = 0.0126$$

$$\lambda_3 = 0.0555$$

1. Все собственные числа положительны. Это математически подтверждает, что система полностью наблюдаема. Нет "слепых зон", куда наша матрица C не смотрит.
2. Разброс значений. Одно собственное число очень большое (604.8), а другое очень маленькое (0.0126). Это значит, что система наблюдаема неравномерно: направление, соответствующее λ_1 , видно отлично (сигнал сильный), а направление, соответствующее λ_2 , видно слабо (сигнал слабый).

Инженер, видя, что Грамиан имеет малые собственные числа при выборе датчика, понимает: нужно ставить более точные (и дорогие) датчики, чтобы "разглядеть" слабые сигналы.

Код для вычисления Грамиана представлен в приложении В (листинг В.4)

Следующим этапом решим обратную задачу: у нас есть только информация с выхода системы $y(t) = f(t)$, но мы не знаем, в каком состоянии система находилась в начальный момент времени. Цель – по "следам" на выходе восстановить "портрет" начального состояния $x(0)$.

Это похоже на работу детектива: видим только результат (выходной сигнал), но должны понять, что было в начале. Если система наблюдаема (а мы это уже доказали в предыдущих пунктах), то такая задача имеет единственное решение.

Начальное состояние вычисляется по формуле:

$$x(0) = (Q(t_1))^{-1} \int_0^{t_1} e^{A^T t} C^T y(t) dt$$

После вычисления получаем:

$$x(0) = \begin{bmatrix} x_1(0) \\ x_2(0) \\ x_3(0) \end{bmatrix} = \begin{bmatrix} -8 \\ -3 \\ 5 \end{bmatrix}$$

На рисунке 5 показан выход системы $y(t)$ и заданный сигнал $f(t)$. Видно, что оба сигнала начинаются с значения около -3 , затем плавно растут и выходят на нулевое

значение к моменту $t = 1$ секунда. После этого сигналы совпадают на всем оставшемся интервале. Такое поведение показывает, что найденное начальное состояние $x(0) = [-8, -3, 5]^T$ корректно воспроизводит выходной сигнал системы.

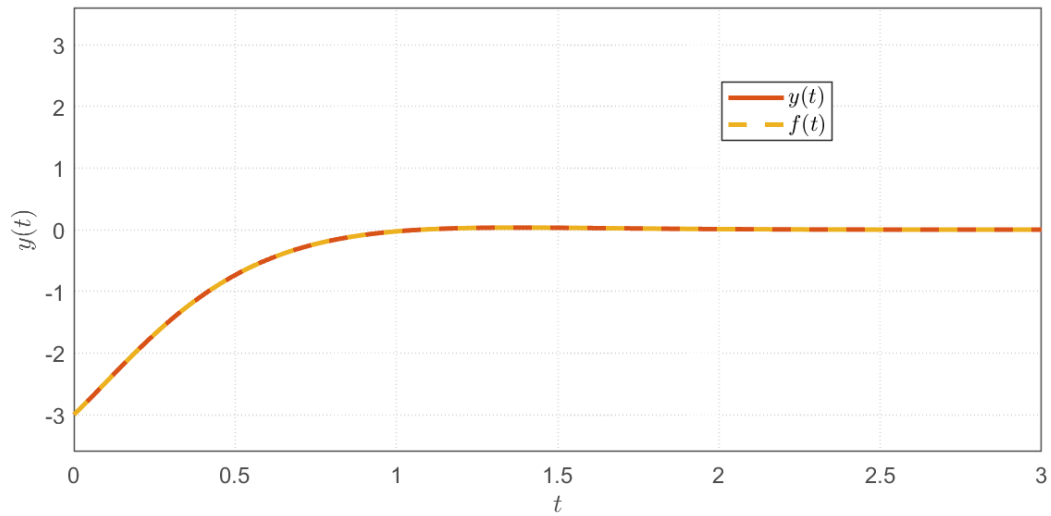


Рисунок 5 — Выход системы $y(t)$ и заданный сигнал $f(t)$

На рисунке 6 показана ошибка $e(t) = y(t) - f(t)$. Видно, что ошибка имеет порядок 10^{-3} , то есть очень мала. В начале интервала ошибка колеблется (достигая примерно $-0.7 \cdot 10^{-3}$), но быстро затухает и к концу интервала становится практически нулевой. Это подтверждает, что вычисления выполнены верно — восстановленное начальное состояние действительно дает выходной сигнал, совпадающий с заданным $f(t)$.

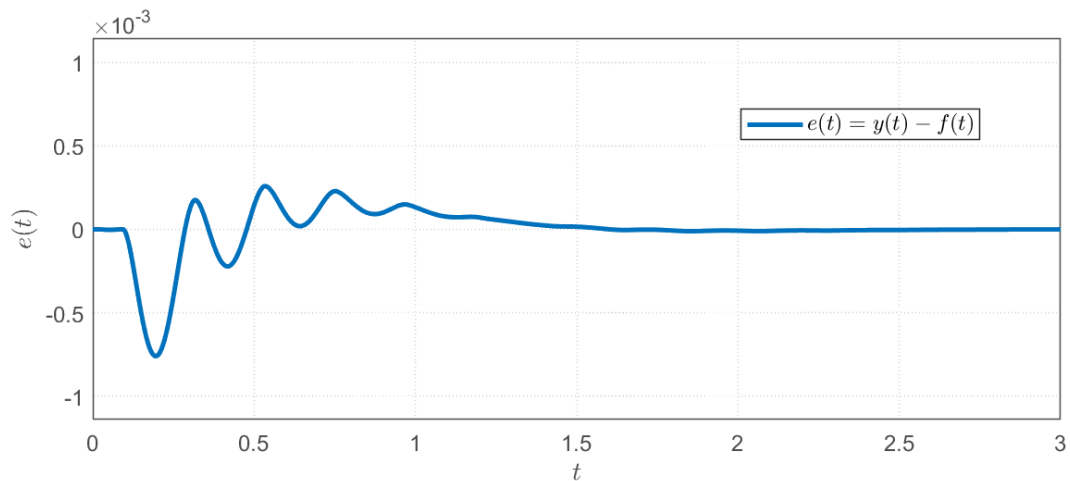


Рисунок 6 — Ошибка $e(t) = y(t) - f(t)$

Итак:

1. Система действительно наблюдаема. Если бы это было не так, мы бы не смогли однозначно восстановить $x(0)$, и графики не совпали бы.
2. Метод Грамиана работает корректно. Формула дала точный результат, что подтверждает правильность теоретических выкладок.

3. Качество восстановления зависит от свойств Грамиана. Помним, что у нас одно собственное число было очень маленьким ($\lambda_3 = 0.0126$). Это значит, что по одному из направлений состояние восстанавливается "хуже". Но в нашем случае погрешность всё равно приемлема.

Восстановление начального состояния – очень важная задача на практике, например, если дрон потерял сигнал GPS на короткое время, по данным с акселерометров и гироскопов можно восстановить, где он находился.

Код для нахождения начальных условий представлен в приложении В (листинг В.5)

3.3 Выводы

1. Система полностью наблюдаема.
2. Грамиан наблюдаемости $Q(t_1)$ при $t_1 = 3$ является невырожденной матрицей, что подтверждает полную наблюдаемость системы.
3. По заданному выходному сигналу $y(t) = f(t) = -3e^{-3t} \cos(2t) - 2e^{-3t} \sin(2t)$ найдены начальные условия:

$$x(0) = [-8; -3; 5]^T.$$

4. Результаты моделирования подтверждают корректность вычислений: выход системы $y(t)$ точно совпадает с заданным сигналом $f(t)$, максимальная ошибка не превышает 10^{-3} .

4 Еще одно исследование наблюдаемости

4.1 Задание

Система описывается системой уравнений:

$$\begin{cases} \dot{x} = Ax \\ y = Cx \end{cases}$$

где:

$$A = \begin{bmatrix} -9 & 0 & -10 \\ -4 & -1 & -6 \\ 6 & -2 & 5 \end{bmatrix}, \quad C = \begin{bmatrix} 0 & -5 & -5 \end{bmatrix}, \quad f(t) = -3e^{-3t} \cos(2t) - 2e^{-3t} \sin(2t)$$

Необходимо:

1. Выполнить все шаги Задания 3 для рассматриваемой системы и выбранной целевой точки x_1 :
 - Исследовать наблюдаемость системы (матрица наблюдаемости, собственные числа, Жорданова форма)
 - Найти Грамиан наблюдаемости для времени $t_1 = 3$, проанализировать его собственные числа
 - Определить начальные условия по выходу $y(t) = f(t)$
 - Выполнить моделирование системы
 2. Определить, мог ли выход $y(t) = f(t)$ быть порождён иными начальными условиями. Если да – привести не менее трёх таких векторов и выполнить моделирование для каждого.
-

4.2 Ход работы

Матрица наблюдаемости определяется по формуле:

$$V = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix}$$

С новыми данными она примет вид:

$$V = \begin{bmatrix} 0 & -5 & -5 \\ -5 & 10 & 20 \\ 40 & 5 & -75 \end{bmatrix}$$

$$\text{rank}(V) = 2$$

Ранг матрицы наблюдаемости меньше размерности системы, значит система не является полностью наблюдаемой.

Код для вывода матрицы наблюдаемости представлен в приложении [Г](#) (листинг [Г.1](#))

Проверим наблюдаемость системы через критерий Хаутуса.

Собственные числа матрицы A известны из предыдущего задания [3](#), проверим управляемость системы с помощью матрицы Хаутуса.

1. Для $\lambda_1 = 1$:

$$H_1 = \begin{bmatrix} 17 & 27 & -7 \\ -6 & -8 & 4 \\ 5 & 11 & 1 \\ 0 & -5 & -5 \end{bmatrix}$$

$$\text{rank}(H_1) = 2$$

2. Для $\lambda_2 = -3 + 2i$:

$$H_2 = \begin{bmatrix} -3 + i & 27 & -7 \\ -6 & -13 + i & 4 \\ 5 & 11 & -4 + i \\ 0 & -5 & -5 \end{bmatrix}$$

$$\text{rank}(H_2) = 3$$

3. Для $\lambda_2 = -3 - 2i$:

$$H_3 = \begin{bmatrix} -3 - i & 27 & -7 \\ -6 & -13 - i & 4 \\ 5 & 11 & -4 - i \\ 0 & -5 & -5 \end{bmatrix}$$

$$\text{rank}(H_3) = 3$$

Последние два собственных числа (комплексные, $\lambda_{1,2} = -3 \pm 2i$) наблюдаемы – их матрицы Хаутуса имеют полный ранг 3.

А вот первое собственное число ($\lambda_3 = 1$) ненаблюдаемо – ранг матрицы H_3 равен 2, а не 3. Это означает, что соответствующий режим системы не виден на выходе при текущей матрице C .

Это полностью согласуется с результатом проверки через матрицу наблюдаемости: $\text{rank}(V) = 2$, то есть система имеет только 2 наблюдаемых направления из 3.

Код программы, реализующий проверку наблюдаемости по критерию Хаутуса, приведен в Приложении Г (листинг Г.2)

Перейдем к вещественной Жордановой форме.

Собственные векторы, матрица перехода P и вещественная жорданова форма J зависят только от матрицы A . Поскольку она совпадает с матрицей из задания 3, данные характеристики остались прежними.

Но вектор наблюдения в новом базисе изменился:

$$\tilde{C} = CP = \begin{bmatrix} 0.7906 & 1.5811 & -0.0000 \end{bmatrix}$$

В новых координатах Жорданова форма запишется следующим образом:

$$\dot{\tilde{x}} = \begin{bmatrix} -3 & 0 & 0 \\ -2 & -3 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tilde{x}, \quad y = \begin{bmatrix} 0.7906 & 1.5811 & -0.0000 \end{bmatrix} \tilde{x}$$

Первые две компоненты матрицы $\tilde{C}$ (0.7906 и 1.5811) относятся к колебательному блоку J_1 . Они ненулевые – колебательная часть системы наблюдаема.

Третья компонента (-0.0000) относится к блоку J_2 . Она нулевая – эта часть НЕ наблюдаема.

Итак, система НЕ полностью наблюдаема.

Код для построения вещественной Жордановой форме представлен в приложении Г (листинг Г.3)

Далее вычислим Грамиан наблюдаемости для времени $t_1 = 3$ секунды.

$$Q(t_1) = \int_0^{t_1} e^{A^T t} C^T C e^{A t} dt$$

Матрица Грамиана примет вид:

$$Q(3) = \begin{bmatrix} 0.6410 & 0.9615 & 1.6026 \\ 0.9615 & 3.5256 & 4.4872 \\ 1.6026 & 4.4872 & 6.0897 \end{bmatrix}$$

Ее собственные числа:

$$\lambda_1 = 9.8497$$

$$\lambda_2 = 0.4068$$

$$\lambda_3 = -0.0000$$

Наличие нулевого собственного числа ($\lambda_3 \approx 0$) указывает на то, что система НЕ полностью наблюдаема. Грамиан наблюдаемости вырожден и имеет ранг 2 вместо 3. Это означает, что одна из трех мод системы не наблюдается на выходе.

Код для построения матрицы Грамиана и нахождения ее собственных чисел представлены в приложении Г (листинг Г.4)

Найдем Начальные условия:

$$x(0) = \begin{bmatrix} 0.0667 \\ 0.2667 \\ 0.3333 \end{bmatrix}$$

На рисунке 7 показано сравнение выходного сигнала системы $y(t)$, полученного при найденных начальных условиях, с заданным сигналом $f(t)$. Как видно из графика, сигналы совпадают, что подтверждает правильность вычислений.

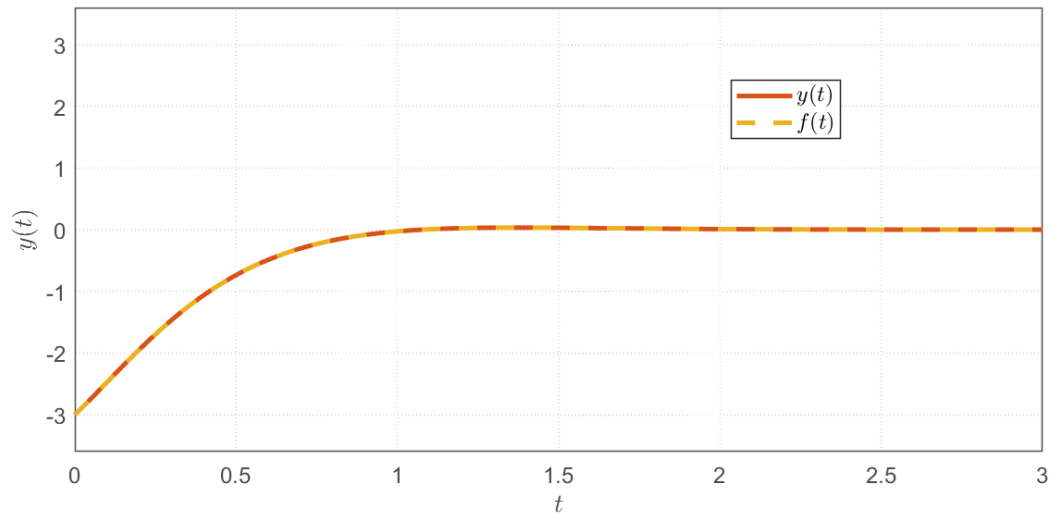


Рисунок 7 — Выход системы $y(t)$ и заданный сигнал $f(t)$

На рисунке 8 представлена ошибка $e(t) = y(t) - f(t)$. Максимальное значение ошибки не превышает 10^{-3} , что свидетельствует о высокой точности восстановления начальных условий. Небольшие расхождения на начальном участке объясняются численными погрешностями интегрирования.

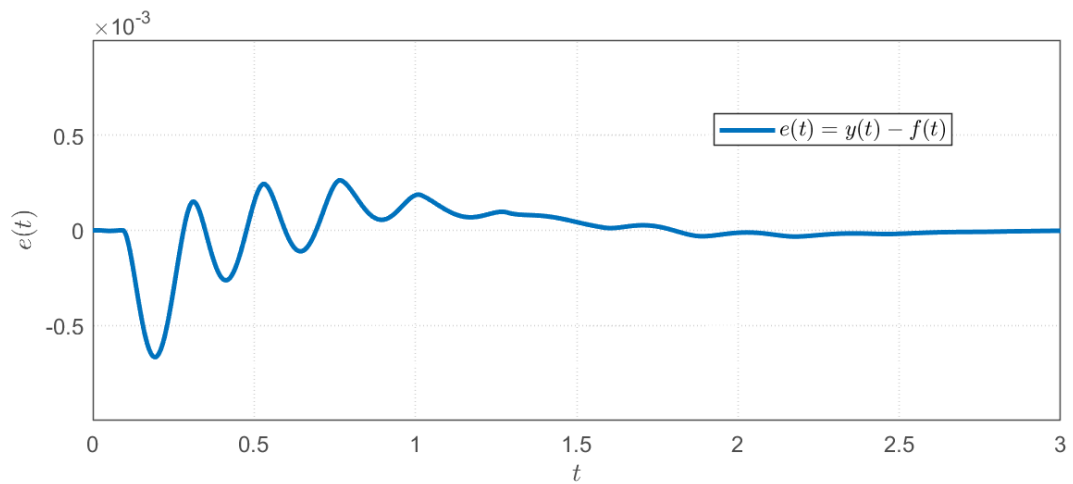


Рисунок 8 — Ошибка $e(t) = y(t) - f(t)$

Код для нахождения начальных условий представлен в приложении Г (листинг Г.5)

Поскольку система оказалась частично наблюдаемой, логично предположить, что один и тот же выходной сигнал может соответствовать разным начальным состояниям. Проверю это предположение.

Ненаблюдаемое подпространство — это множество всех векторов v , для которых $Vv = 0$, то есть ядро матрицы наблюдаемости.

С помощью команды `null(V)` в MATLAB нахожу нормированный базисный вектор ядра:

$$\vec{e} = \begin{bmatrix} -0.5774 \\ -0.5774 \\ 0.5774 \end{bmatrix}$$

Любой вектор начальных условий $\tilde{x}(0)$, при котором выход системы будет таким же $y(t) = f(t)$, как и для найденного ранее $x(0)$, можно записать в виде суммы исходного вектора начальных условий и произвольного вектора из ненаблюдаемого подпространства:

$$\tilde{x}(0) = x(0) + \alpha \cdot \vec{e}, \quad \alpha \in \mathbb{R}$$

где $\vec{e} = [-0.5774; -0.5774; 0.5774]^T$ — нормированный базисный вектор ядра матрицы наблюдаемости, $x(0) = [0.0667; 0.2667; 0.3333]^T$ — начальные условия, найденные с помощью Грамиана наблюдаемости и псевдообратной матрицы (листинг Г.4 в приложении Г).

Результаты вычислений для четырех различных начальных состояний приведены в таблице 1.

Таблица 1 — Различные начальные условия, дающие одинаковый выход

№	α	$x_1(0)$	$x_2(0)$	$x_3(0)$
1	0	0.0667	0.2667	0.3333
2	1	-0.5107	-0.3107	0.9107
3	2	-1.0880	-0.8880	1.4880
4	-1	0.6440	0.8440	-0.2440

На рисунках 9–11 показано изменение компонент вектора состояния $x_1(t)$, $x_2(t)$, $x_3(t)$ для каждого из четырех начальных условий. Как видно из графиков, траектории по каждой компоненте существенно различаются.

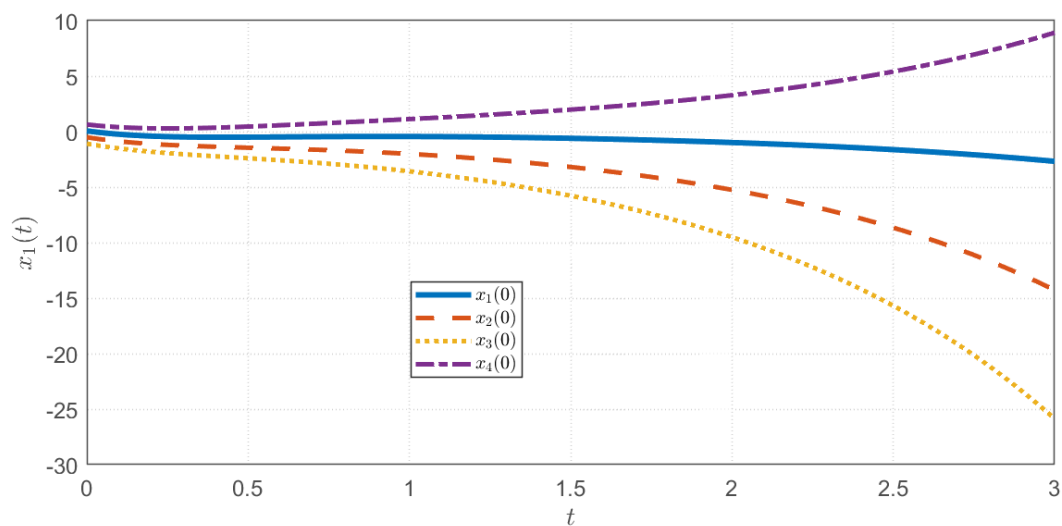


Рисунок 9 — Первая компонента вектора состояния $x_1(t)$ для различных начальных условий

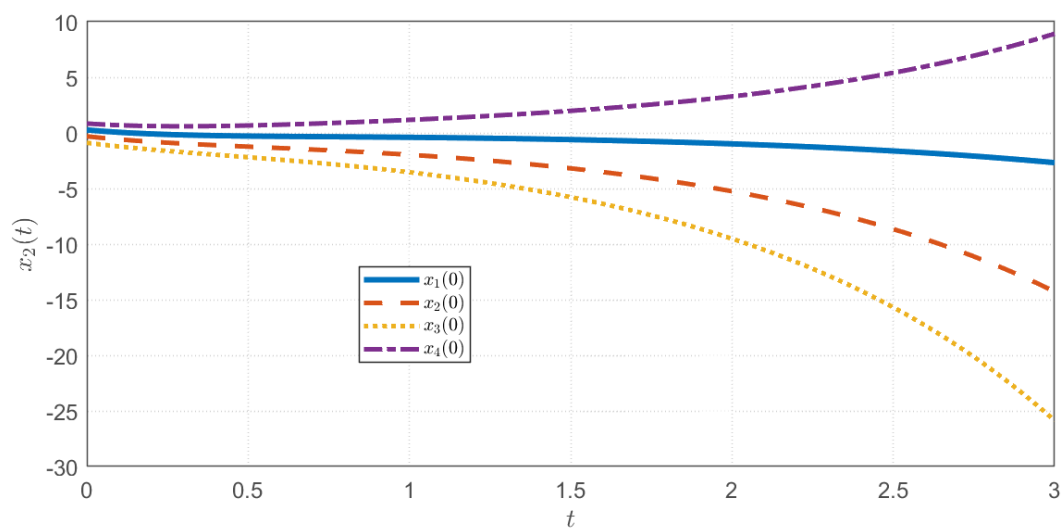


Рисунок 10 — Компонента $x_2(t)$ для различных начальных условий

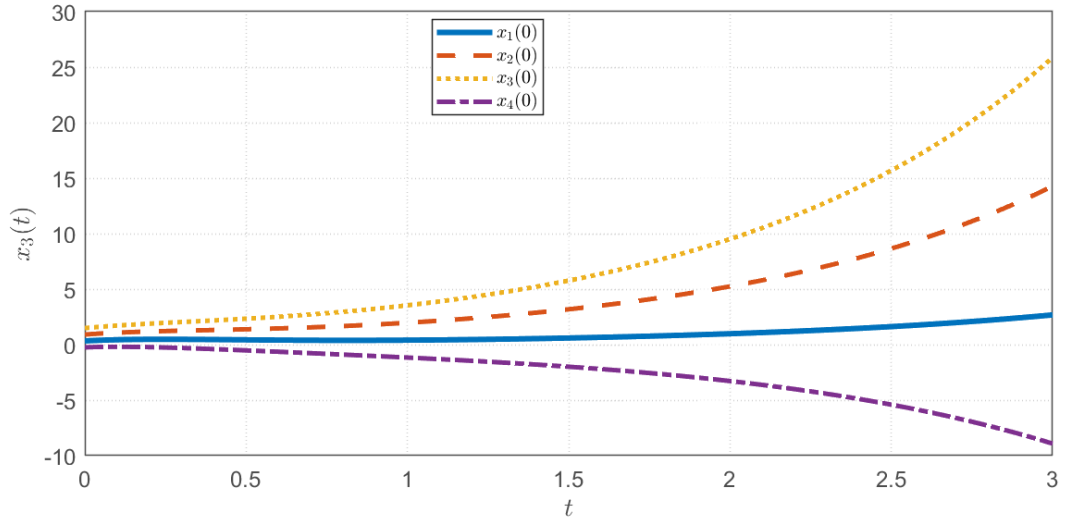


Рисунок 11 — Компонента $x_3(t)$ для различных начальных условий

На рисунке 12 представлен выход системы $y(t)$ для всех четырех случаев в сравнении с заданным сигналом $f(t)$. Все четыре кривые практически совпадают друг с другом и с теоретическим сигналом, что подтверждает правильность вычислений.

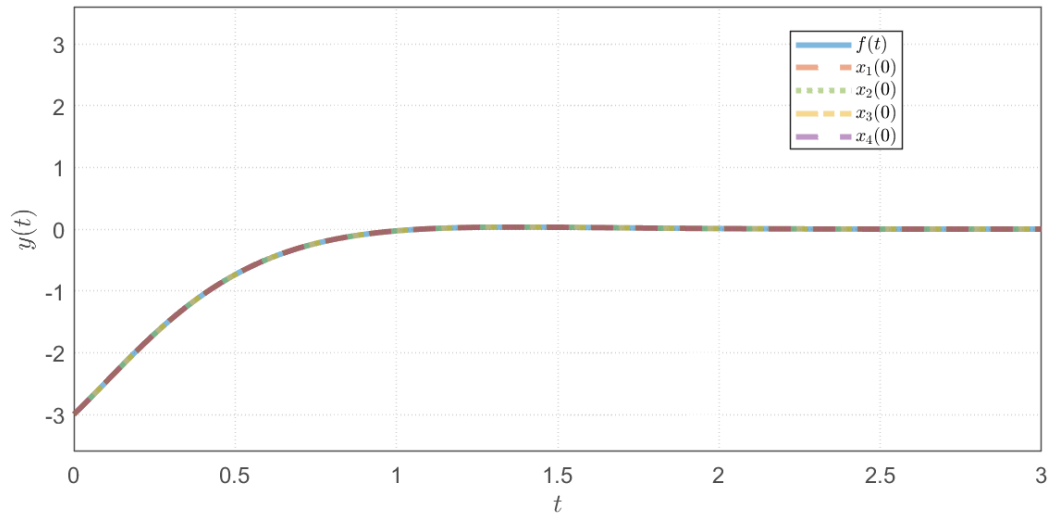


Рисунок 12 — Выход системы $y(t)$ для различных начальных условий

Таким образом, показано, что различные начальные состояния могут порождать одинаковый выходной сигнал, что свидетельствует о наличии ненаблюдаемого подпространства в системе.

Код, вычисляющий альтернативные начальные условия представлен в в приложении Г (листинг Г.6)

4.3 Выводы

1. Система частично наблюдаема (ранг $V = 2$ при $n = 3$).

2. Обнаружена ненаблюдаемая неустойчивая мода ($\lambda_1 = 1$).
3. Грамиан наблюдаемости вырожден (одно собственное число ≈ 0).
4. Начальные условия восстанавливаются неединственно — существует бесконечное множество $x(0)$, дающих одинаковый выход.
5. Моделирование подтвердило теоретические результаты: разные начальные условия порождают одинаковый выходной сигнал.

5 Исследование управляемости по выходу

5.1 Задание

Рассматривается система:

$$\begin{cases} \dot{x} = Ax + Bu \\ y = Cx + Du \end{cases}$$

где

$$A = \begin{bmatrix} 1 & 2 & -1 \\ -6 & -7 & -2 \\ 6 & 6 & 1 \end{bmatrix}, B = \begin{bmatrix} -2 \\ 4 \\ -2 \end{bmatrix}, C = \begin{bmatrix} 0 & 1 & -1 \\ 0 & -3 & 3 \end{bmatrix}$$

Необходимо:

1. Найти вещественную Жорданову форму системы
2. Определить управляемость и наблюдаемость каждого из собственных чисел и системы в целом
3. Найти матрицу управляемости системы по выходу при $D = 0_{2 \times 1}$, определить ее ранг, сделать вывод об управляемости системы по выходу.
4. Проанализировать зависимость управляемости по выходу от (не)управляемости и (не)наблюдаемости собственных чисел.
5. Если система была не полностью управляемой по выходу, то предложить такую матрицу связи D , которая могла бы обеспечить полную управляемость по выходу.

5.2 Ход работы

Собственные векторы, матрица перехода P и вещественная жорданова форма J зависят только от матрицы A . Поскольку она совпадает с матрицей из задания 1, данные характеристики остались прежними.

$$J = \begin{bmatrix} -2 & 3 & 0 \\ -3 & -2 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

Но после перехода к Жорданову базису матрицы B и C преобразуются:

$$\tilde{B} = P^{-1}B = \begin{bmatrix} 3.1623 \\ -3.1623 \\ 2.8284 \end{bmatrix}$$

$$\tilde{C} = CP = \begin{bmatrix} 1.2649 & 0 & 0.7071 \\ -3.7947 & 0 & -2.1213 \end{bmatrix}$$

Исходная система $\dot{x} = Ax + Bu$, $y = Cx + Du$ в Жордановом базисе принимает вид:

$$\dot{\tilde{x}} = J_R \tilde{x} + \tilde{B}u$$

$$y = \tilde{C}\tilde{x} + Du$$

или в развернутой форме:

$$\begin{cases} \dot{\tilde{x}} = \begin{bmatrix} -2 & 3 & 0 \\ -3 & -2 & 0 \\ 0 & 0 & -1 \end{bmatrix} \tilde{x} + \begin{bmatrix} 3.1623 \\ -3.1623 \\ 2.8284 \end{bmatrix} u \\ y = \begin{bmatrix} 1.2649 & 0 & 0.7071 \\ -3.7947 & 0 & -2.1213 \end{bmatrix} \tilde{x} + Du \end{cases}$$

Из матрицы $\tilde{B}$ в Жордановом базисе видно:

$$\tilde{B} = \begin{bmatrix} 3.1623 \\ -3.1623 \\ 2.8284 \end{bmatrix}$$

Первые две строки соответствуют блоку J_1 (комплексная пара $\lambda_{2,3} = -2 \pm 3i$). Обе строки ненулевые – управляемы

Третья строка соответствует блоку J_2 ($\lambda_1 = -1$). Элемент $2.8284 \neq 0$ – управляемо

Все собственные числа управляемы – система полностью управляема.

Из матрицы $\tilde{C}$ в Жордановом базисе:

$$\tilde{C} = \begin{bmatrix} 1.2649 & 0 & 0.7071 \\ -3.7947 & 0 & -2.1213 \end{bmatrix}$$

Блоку J_1 соответствуют первые два столбца. Второй столбец нулевой, но первый

ненулевой – наблюдаемы

Блоку J_2 соответствует третий столбец. Оба элемента ненулевые – наблюдаемо

Все собственные числа наблюдаемы – система полностью наблюдаема.

Код, вычисляющий Жорданову форму, проверяет наблюдаемость и управляемость системы представлен в приложении [Д](#) (листинг [Д.1](#))

При $D = 0_{2 \times 1}$ матрица управляемости по выходу вычисляется по формуле:

$$W_y = \begin{bmatrix} CB & CAB & CA^2B \end{bmatrix}$$

Матрица управляемости по выходу примет вид:

$$W_y = \begin{bmatrix} 6 & -22 & 30 \\ -18 & 66 & -90 \end{bmatrix}$$
$$\text{rank}(W_y) = 1$$

Так как $\text{rank}(W_y) = 1$, то есть меньше числа выходов системы, система не управляема по выходу.

Код, вычисляющий матрицу управляемости по выходу представлен в приложении [Д](#) (листинг [Д.2](#))

Управляемость по выходу не определяется напрямую управляемостью и наблюдаемостью системы. Система может быть полностью управляемой и наблюдаемой по состоянию, но при этом не быть управляемой по выходу.

В рассматриваемой системе матрица выхода имеет вид:

$$C = \begin{bmatrix} 0 & 1 & -1 \\ 0 & -3 & 3 \end{bmatrix}.$$

Ее строки линейно зависимы, поскольку вторая строка равна первой, умноженной на -3 . Следовательно,

$$y_2(t) = -3y_1(t),$$

то есть второй выход не содержит новой информации и фактически дублирует первый.

В результате матрица управляемости по выходу

$$W_y = \begin{bmatrix} 6 & -22 & 30 \\ -18 & 66 & -90 \end{bmatrix}$$

имеет ранг

$$\text{rank}(W_y) = 1 < p = 2,$$

поэтому система не является полностью управляемой по выходу.

Для обеспечения полной управляемости по выходу можно выбрать ненулевую матрицу прямой связи

$$D = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Тогда матрица управляемости по выходу принимает вид

$$W_y = \begin{bmatrix} 0 & 6 & -22 & 30 \\ 1 & -18 & 66 & -90 \end{bmatrix},$$

а ее ранг становится равным двум:

$$\text{rank}(W_y) = 2.$$

Так как ранг матрицы равен числу выходов, система становится полностью управляемой по выходу.

Результаты получены с помощью MATLAB-скрипта (см. листинг [Д.3](#)), реализующего вычисление матрицы управляемости по выходу.

5.3 Выводы

1. Жорданова форма найдена.
2. Система полностью управляема ($\text{rank}(U) = 3$).
3. Система полностью наблюдаема ($\text{rank}(V) = 3$).
4. Система не управляема по выходу при $D = 0$ ($\text{rank}(Q_{\text{out}}) = 1 < 2$). Причина — строки матрицы C линейно зависимы ($C_2 = -3C_1$), поэтому оба выхода y_1 и y_2 реагируют на управление одинаково.
5. Полная управляемость и наблюдаемость системы не гарантируют управляемость по выходу. Последняя зависит еще и от структуры матриц C и D .
6. Предложена матрица $D = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, при которой система становится управляемой по выходу ($\text{rank}(W_y) = 2$).

6 Вывод

В ходе выполнения лабораторной работы я подробно разобралась с понятиями управляемости и наблюдаемости линейных систем.

Основные итоги работы:

- Я научилась проверять управляемость и наблюдаемость разными способами (через матрицы, критерий Хаутуса, Жорданову форму и Грамианы). Поняла, что все эти методы работают в связке и подтверждают друг друга.
- Увидела на практике, чем отличается полная управляемость от частичной. Если система "не дослушивается" или "не досматривает" какие-то режимы, это нужно учитывать: выбирать только достижимые точки или понимать, что часть состояния скрыта от датчиков.
- Научилась решать две главные практические задачи: находить управляющий сигнал, чтобы перевести систему в нужную точку, и восстанавливать начальные условия по известному выходному сигналу.
- Поняла физический смысл Грамианов. Они не просто отвечают "да или нет" но и показывают, насколько легко управлять системой и насколько точно мы можем восстановить ее состояние.
- Самым интересным для меня стало пятое задание. Я убедилась, что система может быть полностью управляема и наблюдаема "внутри" но при этом не управляема по выходу, если выходы дублируют друг друга. И главное – эту проблему можно решить, просто подобрав нужную матрицу D .

В итоге я не просто посчитала формулы в MATLAB, а реально поняла, как эти математические свойства влияют на то, можно ли вообще управлять реальным объектом и что для этого нужно.

Приложение А

Листинг А.1 — Вычисление матрицы наблюдаемости

```
1 % матрицы
2 A = [1 2 -1; -6 -7 -2; 6 6 1];
3
4 B = [-2; 4; -2];
5
6 % построение матрицы управляемости
7 Co = ctrb(A, B);
8
9 % вывод матрицы управляемости
10 disp('Матрица управляемости Co = [B AB A^2B:]');
11 disp(Co);
12
13 % проверка ранга
14 r = rank(Co);
15 n = size(A,1);
16
17 fprintf('Ранг матрицы управляемости = %d\n', r);
18
19 % проверка управляемости
20 if r == n
21     disp('-> Система ПОЛНОСТЬЮ УПРАВЛЯЕМА');
22 else
23     disp('-> Система НЕ полностью управляема');
24 end
```

Листинг А.2 — Вычисление собственных чисел и матриц Хаутуса

```
1 % матрицы
2 A = [1 2 -1; -6 -7 -2; 6 6 1];
3
4 B = [-2; 4; -2];
5
6 % собственные числа
7 lambda = eig(A);
8
9 disp('Собственные числа матрицы A:');
10 disp(lambda);
11
12 n = size(A,1);
13
14 % массив для рангов
15 r = zeros(length(lambda),1);
16
17 for i = 1:length(lambda)
18
19     H = [lambda(i)*eye(n) - A, B];
```



```

20
21     fprintf('\nМатрица Хаутуса для lambda_%d:\n', i);
22     disp(H);
23
24     r(i) = rank(H);
25
26     fprintf('Ранг H_%d = %d\n', i, r(i));
27
28     if r(i) == n
29         disp('-> Собственное число управляемо');
30     else
31         disp('-> Собственное число НЕ управляемо');
32     end
33
34 end
35
36 % итоговый вывод по системе
37 disp('-----');
38
39 if all(r == n)
40     disp('СИСТЕМА ПОЛНОСТЬЮ УПРАВЛЯЕМА ПО КРИТЕРИЮ ХАУТУСА');
41 else
42     disp('СИСТЕМА НЕ ПОЛНОСТЬЮ УПРАВЛЯЕМА ПО КРИТЕРИЮ ХАУТУСА');
43 end

```

Листинг А.3 — Расчет вещественной Жордановой формы

```

1  clc; clear;
2  % матрицы
3  A = [1 2 -1; -6 -7 -2; 6 6 1];
4  B = [-2; 4; -2];
5
6  % собственные числа и векторы
7  [V, D] = eig(A);
8  lambda = diag(D); % столбец собственных чисел
9
10 % берем векторы из столбцов v
11 v_c = V(:,1); % комплексный вектор
12 v_r = V(:,3); % вещественный вектор
13
14 % разбиваем комплексный вектор на вещественную и мнимую части
15 p1 = real(v_c);
16 p2 = imag(v_c);
17 p3 = v_r;
18
19 % матрица перехода P
20 P = [p1, p2, p3];
21
22 % обратная матрица
23 P_inv = inv(P);

```

```

24
25 % жорданова форма J = P^-1 * A * P
26 J = P_inv * A * P;
27
28 % вектор управления в новом базисе
29 B_tilde = P_inv * B;
30
31 % вывод результатов
32 disp('--- Результаты ---');
33 disp('Матрица P:'); disp(P);
34 disp('Матрица P^-1:'); disp(P_inv);
35 disp('Жорданова форма J:'); disp(J);
36 disp('Вектор B_tilde:'); disp(B_tilde);
37
38 % проверка управляемости по строкам B_tilde
39 disp('--- Проверка управляемости ---');
40 if B_tilde(1) ~= 0 || B_tilde(2) ~= 0
41     disp('Блок J1 (комплексные корни) -> УПРАВЛЯЕМ');
42 else
43     disp('Блок J1 (комплексные корни) -> НЕ управляем');
44 end
45
46 if B_tilde(3) ~= 0
47     disp('Блок J2 (вещественный корень) -> УПРАВЛЯЕМ');
48 else
49     disp('Блок J2 (вещественный корень) -> НЕ управляем');
50 end

```

Листинг А.4 — Вычисление Грамиана

```

1 % матрицы
2 A = [1 2 -1; -6 -7 -2; 6 6 1];
3 B = [-2; 4; -2];
4 t1 = 3;
5
6 % вычисление Грамиана
7 % P = int_0^t1 exp(A*t)*B*B'*exp(A'*t) dt
8 gramian_fun = @(t) expm(A*t) * B * B' * expm(A'*t);
9 W = integral(gramian_fun, 0, t1, 'ArrayValued', true);
10
11 disp('=== Грамиан управляемости W ===');
12 disp(W);
13
14 % собственные числа Грамиана
15 ev = eig(W);
16 disp('Собственные числа Грамиана:');
17 disp(ev);
18
19 % простая проверка знака
20 if all(ev > 0)

```

```

21     disp('-> Все собственные числа > 0: система полностью управляема');
22 else
23     disp('-> Внимание: есть нулевые или отрицательные числа');
24 end

```

Листинг A.5 — Поиск управления для перевода в точку x_1

```

1  clear; clc; close all;
2
3  %% исходные данные
4  A = [1  2 -1; -6 -7 -2; 6  6  1];
5  B = [2; 0; 0];
6  x1 = [ 2; -3; 3];
7  t1 = 3;
8
9  %% матрица управляемости
10 U = ctrb(A,B);
11 rank_U = rank(U);
12 disp('=== 1. Матрица управляемости ===')
13 disp('U =')
14 disp(U)
15 disp(['rank(U) = ', num2str(rank_U)])
16
17 %% собственные числа
18 disp('=== 2. Собственные числа матрицы A ===')
19 eig_A = eig(A);
20 disp(eig_A)
21
22 %% грамиан управляемости
23 disp('=== 3. Грамиан P(t1) и обратная матрица ===')
24 N = 1000;
25 dt = t1/N;
26 Wc = zeros(3);
27
28 for k = 0:N
29     tau = k*dt;
30     exp_At = expm(A*tau);
31     Wc = Wc + exp_At * B * B' * exp_At' * dt;
32 end
33
34 disp('P(t1) = Wc =')
35 disp(Wc)
36
37 disp('Обратная матрица P(t1)^(-1) =')
38 disp(pinv(Wc))
39
40 disp('Собственные числа Грамиана:')
41 disp(eig(Wc))
42
43 %% управляющее воздействие

```

```

44 Wc_pinv = pinv(Wc);
45
46 t_span = 0:0.01:t1;
47 u_num = zeros(size(t_span));
48
49 for k = 1:length(t_span)
50     tau = t_span(k);
51     exp_At = expm(A'*(t1 - tau));
52     u_num(k) = B' * exp_At * Wc_pinv * x1;
53 end
54
55 %% моделирование системы
56 [t_sol, x_sol] = ode45(@(t,x) A*x + ...
57     B*interp1(t_span, u_num, t, 'linear', 0), ...
58     t_span, [0; 0; 0]);
59
60 %% проверка конечного состояния
61 disp('=== 6. Проверка результата ===')
62 x_final = x_sol(end, :);
63 disp('Полученное состояние x(t1):')
64 disp(x_final)
65 disp('Целевое состояние x1:')
66 disp(x1)
67
68 %% параметры оформления
69 colors = lines(7);
70 T = 12; M = 14; L = 12;
71
72 %% график управляющего воздействия u(t)
73 figure('Color','white','Position',[100 100 900 400]); hold on
74 plot(t_span, u_num, 'LineWidth', 2.5, 'Color', colors(1,:)) % СИНИЙ цвет
75
76 % пунктирные направляющие линии для u(t)
77 xline(t1, '--k', 'LineWidth', 1, 'HandleVisibility', 'off');
78
79 ax = gca;
80 ax.LineWidth = 0.8;
81 ax.FontSize = T;
82 ax.XColor = [0.3 0.3 0.3];
83 ax.YColor = [0.3 0.3 0.3];
84 ax.TickLength = [0 0];
85 ax.TickDir = 'out';
86
87 grid on;
88 box on;
89 ax.GridLineStyle = ':';
90 ax.GridColor = [0.7 0.7 0.7];
91 ax.GridAlpha = 0.6;
92

```

```

93 xlim([0, t1 + 0.3]);
94
95 xlabel('$t$', 'Interpreter', 'latex', 'FontSize', M)
96 ylabel('$u(t)$', 'Interpreter', 'latex', 'FontSize', M)
97 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
98
99 %% графики компонент вектора состояния x(t)
100 figure('Color','white','Position',[100 100 900 400]); hold on
101 plot(t_sol, x_sol(:,1), 'LineWidth', 2.5, 'Color', colors(1,:))
102 plot(t_sol, x_sol(:,2), 'LineWidth', 2.5, 'Color', colors(2,:))
103 plot(t_sol, x_sol(:,3), 'LineWidth', 2.5, 'Color', colors(3,:))
104
105 % пунктирные направляющие линии для x(t)
106 xline(t1, '--k', 'LineWidth', 1, 'HandleVisibility', 'off');
107 yline(x1(1), '--k', 'LineWidth', 1, 'HandleVisibility', 'off');
108 yline(x1(2), '--k', 'LineWidth', 1, 'HandleVisibility', 'off');
109 yline(x1(3), '--k', 'LineWidth', 1, 'HandleVisibility', 'off');
110
111 ax = gca;
112 ax.LineWidth = 0.8;
113 ax.FontSize = T;
114 ax.XColor = [0.3 0.3 0.3];
115 ax.YColor = [0.3 0.3 0.3];
116 ax.TickLength = [0 0];
117 ax.TickDir = 'out';
118
119 grid on;
120 box on;
121 ax.GridLineStyle = ':';
122 ax.GridColor = [0.7 0.7 0.7];
123 ax.GridAlpha = 0.6;
124
125 xlim([0, t1 + 0.3]);
126
127 xlabel('$t$', 'Interpreter', 'latex', 'FontSize', M)
128 ylabel('$x(t)$', 'Interpreter', 'latex', 'FontSize', M)
129 legend('$x_1$', '$x_2$', '$x_3$', ...
130        'Interpreter', 'latex', 'Location', 'best', 'FontSize', L);
131 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
132

```

Приложение Б

Листинг Б.1 — Расчет вещественной Жордановой формы

```
1 % матрицы
2 A = [1 2 -1; -6 -7 -2; 6 6 1];
3 B = [2; 0; 0]; % <- новая матрица B
4
5 % проверяемые точки
6 x1_prime = [2; -3; 3];
7 x1_double_prime = [1; -2; 3];
8
9 % матрица управляемости
10 U = ctrb(A, B);
11 rank_U = rank(U);
12
13 disp('== Матрица управляемости U ==');
14 disp('U = [B A*B A^2*B] =');
15 disp(U);
16 disp(['rank(U) = ', num2str(rank_U)]);
17 disp(' ');
18
19 disp('== Проверка принадлежности точек ==');
20
21 % проверка первой точки
22 U_x1_prime = [U x1_prime];
23 rank_1 = rank(U_x1_prime);
24 disp('Точка x1'' = [2; -3; 3]:');
25 disp(' [U x1''] = ');
26 disp(U_x1_prime);
27 disp(['rank([U x1'']) = ', num2str(rank_1)]);
28 if rank_1 == rank_U
29     disp('-> ПРИНАДЛЕЖИТ управляемому подпространству');
30 else
31     disp('-> НЕ принадлежит');
32 end
33 disp(' ');
34
35 % проверка второй точки
36 U_x1_double_prime = [U x1_double_prime];
37 rank_2 = rank(U_x1_double_prime);
38 disp('Точка x1''' = [1; -2; 3]:');
39 disp(' [U x1''''] = ');
40 disp(U_x1_double_prime);
41 disp(['rank([U x1'''']) = ', num2str(rank_2)]);
42 if rank_2 == rank_U
43     disp('-> ПРИНАДЛЕЖИТ управляемому подпространству');
44 else
45     disp('-> НЕ принадлежит');
```

```

46 end
47 disp(' ');
48
49 % выбор целевой точки
50 if rank_1 == rank_U
51     x1 = x1_prime;
52     disp('Выбрана целевая точка: x1 = x1''');
53     disp(x1);
54 elseif rank_2 == rank_U
55     x1 = x1_double_prime;
56     disp('Выбрана целевая точка: x1 = x1''''');
57     disp(x1);
58 else
59     disp('ОШИБКА: Ни одна точка не принадлежит!');
60 end

```

Листинг Б.2 — Вычисление собственных чисел и матриц Хаутуса

```

1 % матрицы
2 A = [1 2 -1; -6 -7 -2; 6 6 1];
3
4 B = [2; 0; 0];
5
6 % собственные числа
7 lambda = eig(A);
8
9 disp('Собственные числа матрицы A:');
10 disp(lambda);
11
12 n = size(A,1);
13
14 % массив для рангов
15 r = zeros(length(lambda),1);
16
17 for i = 1:length(lambda)
18
19     H = [lambda(i)*eye(n) - A, B];
20
21     fprintf('\nМатрица Хаутуса для lambda_%d:\n', i);
22     disp(H);
23
24     r(i) = rank(H);
25
26     fprintf('Ранг H_%d = %d\n', i, r(i));
27
28     if r(i) == n
29         disp('-> Собственное число управляемо');
30     else
31         disp('-> Собственное число НЕ управляемо');
32     end

```

```

33
34 end
35
36 % итоговый вывод по системе
37 disp('-----');
38
39 if all(r == n)
40     disp('СИСТЕМА ПОЛНОСТЬЮ УПРАВЛЯЕМА ПО КРИТЕРИЮ ХАУТУСА');
41 else
42     disp('СИСТЕМА НЕ ПОЛНОСТЬЮ УПРАВЛЯЕМА ПО КРИТЕРИЮ ХАУТУСА');
43 end

```

Листинг Б.3 — Вычисление собственных чисел и матриц Хаутуса

```

1  clc; clear;
2  % матрицы
3  A = [1 2 -1; -6 -7 -2; 6 6 1];
4  B = [2; 0; 0];
5
6  % собственные числа и векторы
7  [V, D] = eig(A);
8  lambda = diag(D); % столбец собственных чисел
9
10 % берем векторы из столбцов v
11 v_c = V(:,1); % комплексный вектор
12 v_r = V(:,3); % вещественный вектор
13
14 % разбиваем комплексный вектор на вещественную и мнимую части
15 p1 = real(v_c);
16 p2 = imag(v_c);
17 p3 = v_r;
18
19 % матрица перехода P
20 P = [p1, p2, p3];
21
22 % обратная матрица
23 P_inv = inv(P);
24
25 % жорданова форма J = P^-1 * A * P
26 J = P_inv * A * P;
27
28 % вектор управления в новом базисе
29 B_tilde = P_inv * B;
30
31 % вывод результатов
32 disp('--- Результаты ---');
33 disp('Матрица P:'); disp(P);
34 disp('Матрица P^-1:'); disp(P_inv);
35 disp('Жорданова форма J:'); disp(J);
36 disp('Вектор B_tilde:'); disp(B_tilde);

```



```

37
38 % проверка управляемости по строкам B_tilde
39 disp('--- Проверка управляемости ---');
40 if B_tilde(1) ~= 0 || B_tilde(2) ~= 0
41     disp('Блок J1 (комплексные корни) -> УПРАВЛЯЕМ');
42 else
43     disp('Блок J1 (комплексные корни) -> НЕ управляем');
44 end
45
46 tolerance = 1e-6; % допуск
47
48 if abs(B_tilde(3)) > tolerance
49     disp('Блок J2 (вещественный корень) -> УПРАВЛЯЕМ');
50 else
51     disp('Блок J2 (вещественный корень) -> НЕ управляем');
52 end

```

Листинг Б.4 — Вычисление Грамиана

```

1 % матрицы
2 A = [1 2 -1; -6 -7 -2; 6 6 1];
3 B = [2; 0; 0];
4 t1 = 3;
5
6 % вычисление Грамиана
7 % P = int_0^t1 exp(A*t)*B*B'*exp(A'*t) dt
8 gramian_fun = @(t) expm(A*t) * B * B' * expm(A'*t);
9 W = integral(gramian_fun, 0, t1, 'ArrayValued', true);
10
11 disp('=== Грамиан управляемости W ===');
12 disp(W);
13
14 % собственные числа Грамиана
15 ev = eig(W);
16 disp('Собственные числа Грамиана:');
17 disp(ev);
18
19 % простая проверка знака
20 if all(ev > 0)
21     disp('-> Все собственные числа > 0: система полностью управляема');
22 else
23     disp('-> Внимание: есть нулевые или отрицательные числа');
24 end

```

Листинг Б.5 — Поиск управления для перевода в точку

```

1 % исходные данные
2 A = [1 2 -1; -6 -7 -2; 6 6 1];
3 B = [-2; 4; -2];
4 x1 = [2; -3; 3];

```

```

5 t1 = 3;
6
7 % матрица управляемости
8 U = ctrb(A, B);
9 rank_U = rank(U);
10 det_U = det(U);
11
12 % собственные числа
13 eig_A = eig(A);
14
15 % грамиан управляемости
16 Wc = lyap(A, B*B');
17
18 % управление
19 syms t
20 u = B' * expm(A' * (t1 - t)) * inv(Wc) * x1;
21
22 % моделирование
23 t_span = 0:0.01:t1;
24 u_num = zeros(size(t_span));
25 for i = 1:length(t_span)
26     u_num(i) = double(subs(u, t, t_span(i)));
27 end
28
29 % решение дифференциального уравнения
30 [t_sol, x_sol] = ode45(@(t,x) A*x + B*interp1(t_span, u_num, t), t_span, [0;
    0; 0]);
31
32 %% параметры оформления
33 colors = lines(7);
34 T = 12; M = 14; L = 12;
35
36 %% график 1: управляющее воздействие u(t)
37 figure('Color','white','Position',[100 100 900 400]); hold on
38 plot(t_span, u_num, 'LineWidth', 2.5, 'Color', colors(1,:))
39
40 xline(t1, '--k', 'LineWidth', 1, 'HandleVisibility', 'off');
41
42 ax = gca;
43 ax.LineWidth = 0.8;
44 ax.FontSize = T;
45 ax.XColor = [0.3 0.3 0.3];
46 ax.YColor = [0.3 0.3 0.3];
47 ax.TickLength = [0 0];
48 ax.TickDir = 'out';
49
50 grid on;
51 box on;
52 ax.GridLineStyle = ':';

```

```

53 ax.GridColor = [0.7 0.7 0.7];
54 ax.GridAlpha = 0.6;
55
56 xlim([0, t1 + 0.3]);
57
58 % автоматические отступы по Y для u(t)
59 u_max = max(abs(u_num));
60 ylim([-1.2*u_max, 1.2*u_max]);
61
62 xlabel('$t$', 'Interpreter', 'latex', 'FontSize', M)
63 ylabel('$u(t)$', 'Interpreter', 'latex', 'FontSize', M)
64 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
65 ;
66
67 %% график 2: компоненты вектора состояния x(t)
68 figure('Color', 'white', 'Position', [100 100 900 400]); hold on
69 plot(t_sol, x_sol(:,1), 'LineWidth', 2.5, 'Color', colors(1,:))
70 plot(t_sol, x_sol(:,2), 'LineWidth', 2.5, 'Color', colors(2,:))
71 plot(t_sol, x_sol(:,3), 'LineWidth', 2.5, 'Color', colors(3,:))
72
73 xline(t1, '--k', 'LineWidth', 1, 'HandleVisibility', 'off');
74 yline(x1(1), '--k', 'LineWidth', 1, 'HandleVisibility', 'off');
75 yline(x1(2), '--k', 'LineWidth', 1, 'HandleVisibility', 'off');
76 yline(x1(3), '--k', 'LineWidth', 1, 'HandleVisibility', 'off');
77
78 ax = gca;
79 ax.LineWidth = 0.8;
80 ax.FontSize = T;
81 ax.XColor = [0.3 0.3 0.3];
82 ax.YColor = [0.3 0.3 0.3];
83 ax.TickLength = [0 0];
84 ax.TickDir = 'out';
85
86 grid on;
87 box on;
88 ax.GridLineStyle = ':';
89 ax.GridColor = [0.7 0.7 0.7];
90 ax.GridAlpha = 0.6;
91
92 xlim([0, t1 + 0.3]);
93
94 % находим максимальное и минимальное значения среди всех x(t)
95 x_max = max(x_sol(:));
96 x_min = min(x_sol(:));
97 margin = 0.3 * max(abs(x_max), abs(x_min)); % Отступ 30%
98 ylim([x_min - margin, x_max + margin]);
99 % =====
100 xlabel('$t$', 'Interpreter', 'latex', 'FontSize', M)

```

```

101 ylabel('$x(t)$', 'Interpreter', 'latex', 'FontSize', M)
102 legend('$x_1$', '$x_2$', '$x_3$', ...
103         'Interpreter', 'latex', 'Location', 'best', 'FontSize', L);
104 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
    ;

```

Приложение В

Листинг В.1 — Вычисление матрицы наблюдаемости

```
1 % матрицы
2 A = [-9 0 -10; -4 -1 -6; 6 -2 5];
3 C = [2 -1 2];
4
5 % построение матрицы наблюдаемости V
6 V = [C; C*A; C*A^2];
7
8 % вывод результатов
9 disp('== Матрица наблюдаемости V ==');
10 disp('V =');
11 disp(V);
12
13 % находим ранг
14 rank_V = rank(V);
15 disp(['rank(V) = ', num2str(rank_V)]);
16
17 % проверка наблюдаемости
18 if rank_V == 3
19     disp('-> Система ПОЛНОСТЬЮ наблюдаема');
20 else
21     disp('-> Система НЕ ПОЛНОСТЬЮ наблюдаема');
22 end
```

Листинг В.2 — Вычисление матриц Хаутуса для наблюдаемости

```
1 % матрицы
2 A = [-9 0 -10; -4 -1 -6; 6 -2 5];
3 C = [2 -1 2];
4
5 % собственные числа
6 lambda = eig(A);
7
8 disp('Собственные числа матрицы A:');
9 disp(lambda);
10
11 n = size(A,1);
12
13 % массив для рангов
14 r = zeros(length(lambda),1);
15
16 for i = 1:length(lambda)
17
18     H = [lambda(i)*eye(n) - A; C];
19
20     fprintf('\nМатрица Хаутуса для lambda_%d:\n', i);
21     disp(H);
22 end
```

```

22
23     r(i) = rank(H);
24
25     fprintf('Ранг H_%d = %d\n', i, r(i));
26
27     if r(i) == n
28         disp('-> Собственное число наблюдаемо');
29     else
30         disp('-> Собственное число НЕ наблюдаемо');
31     end
32
33 end
34
35 % итоговый вывод по системе
36 disp('-----');
37
38 if all(r == n)
39     disp('СИСТЕМА ПОЛНОСТЬЮ НАБЛЮДАЕМА ПО КРИТЕРИЮ ХАУТУСА');
40 else
41     disp('СИСТЕМА НЕ ПОЛНОСТЬЮ НАБЛЮДАЕМА ПО КРИТЕРИЮ ХАУТУСА');
42 end

```

Листинг В.3 — Вычисление Жордановой формы

```

1 % матрицы
2 A = [-9 0 -10; -4 -1 -6; 6 -2 5];
3 C = [2 -1 2];
4
5 % собственные числа и векторы
6 [V, D] = eig(A);
7 lambda = diag(D); % столбец собственных чисел
8
9 disp('Собственные числа:');
10 disp(lambda);
11
12 % ищем комплексный и вещественный векторы
13 v_c = []; % комплексный вектор
14 v_r = []; % вещественный вектор
15
16 for k = 1:3
17     if abs(imag(lambda(k))) > 0.01 && isempty(v_c)
18         v_c = V(:, k); % комплексный вектор
19     elseif abs(imag(lambda(k))) < 0.01 && isempty(v_r)
20         v_r = real(V(:, k)); % вещественный вектор
21     end
22 end
23
24 % разбиваем комплексный вектор на вещественную и мнимую части
25 p1 = real(v_c);
26 p2 = imag(v_c);

```

```

27 p3 = v_r;
28
29 % матрица перехода P
30 P = [p1, p2, p3];
31
32 % проверяем, не вырождена ли P
33 if abs(det(P)) < 1e-10
34     disp('ВНИМАНИЕ: Матрица P вырождена! Меняем порядок столбцов...');
35     P = [p3, p1, p2]; % другой порядок
36 end
37
38 disp('Матрица P:'); disp(P);
39 disp('Определитель P:'); disp(det(P));
40
41 % обратная матрица
42 P_inv = inv(P);
43
44 % жорданова форма J = P^-1 * A * P
45 J = P_inv * A * P;
46
47 % матрица выхода в новом базисе
48 C_tilde = C * P;
49
50 % вывод результатов
51 disp('--- Результаты ---');
52 disp('Жорданова форма J:'); disp(J);
53 disp('Вектор C_tilde:'); disp(C_tilde);
54
55 % проверка наблюдаемости по строкам C_tilde
56 disp('--- Проверка наблюдаемости ---');
57 tolerance = 1e-6; % допуск
58
59 if abs(C_tilde(1)) > tolerance || abs(C_tilde(2)) > tolerance
60     disp('Блок J1 (комплексные корни) -> НАБЛЮДАЕМ');
61 else
62     disp('Блок J1 (комплексные корни) -> НЕ наблюдаем');
63 end
64
65 if abs(C_tilde(3)) > tolerance
66     disp('Блок J2 (вещественный корень) -> НАБЛЮДАЕМ');
67 else
68     disp('Блок J2 (вещественный корень) -> НЕ наблюдаем');
69 end

```

Листинг В.4 — Вычисление Грамиана

```

1 % матрицы
2 A = [-9 0 -10; -4 -1 -6; 6 -2 5];
3 C = [2 -1 2];
4 t1 = 3;

```

```

5
6 % подынтегральная функция: exp(A'*t) * C' * C * exp(A*t)
7 integrand = @(t) expm(A'*t) * C' * C * expm(A*t);
8
9 % численное интегрирование от 0 до t1
10 Q = integral(integrand, 0, t1, 'ArrayValued', true);
11
12 % вывод результатов
13 disp('== Грамиан наблюдаемости Q(t1) ==');
14 disp('Q =');
15 disp(Q);
16
17 % собственные числа
18 eig_Q = eig(Q);
19 disp('Собственные числа Грамиана:');
20 disp(eig_Q);
21
22 % анализ
23 if all(eig_Q > 0)
24     disp('-> Все собственные числа > 0: система ПОЛНОСТЬЮ наблюдаема');
25 else
26     disp('-> Внимание: есть нулевые или отрицательные числа');
27 end

```

Листинг В.5 — Нахождение начальных условий

```

1 %% исходные данные
2 A = [-9 0 -10; -4 -1 -6; 6 -2 5];
3 C = [2 -1 2];
4 t1 = 3;
5
6 f = @(t) -3*exp(-3*t).*cos(2*t) - 2*exp(-3*t).*sin(2*t);
7
8 %% вычисление Грамиана наблюдаемости
9 Q = integral(@(t) expm(A'*t)*C'*C*expm(A*t), 0, t1, 'ArrayValued', true);
10
11 %% вычисление начальных условий
12 I = integral(@(t) expm(A'*t)*C'*f(t), 0, t1, 'ArrayValued', true);
13 x0 = inv(Q)*I;
14
15 disp('Начальные условия x(0):');
16 disp(x0);
17
18 %% решение ОДУ
19 tspan = linspace(0, t1, 1000);
20 [t, x] = ode45(@(t, x) A*x, tspan, x0);
21
22 %% вычисление выхода
23 y = (C*x')';
24 y_ref = f(t);

```



```

25
26 %% параметры оформления
27 colors = lines(7);
28 T = 12; M = 14; L = 12;
29
30 %% график  $y(t)$  и  $f(t)$ 
31 figure('Color','white','Position',[100 100 900 400]); hold on
32 plot(t, y, 'LineWidth', 2.5, 'Color', colors(2,:))
33 plot(t, y_ref, '--', 'LineWidth', 2.5, 'Color', colors(3,:))
34
35 ax = gca;
36 ax.LineWidth = 0.8;
37 ax.FontSize = T;
38 ax.XColor = [0.3 0.3 0.3];
39 ax.YColor = [0.3 0.3 0.3];
40 ax.TickLength = [0 0];
41 ax.TickDir = 'out';
42
43 grid on;
44 box on;
45 ax.GridLineStyle = ':';
46 ax.GridColor = [0.7 0.7 0.7];
47 ax.GridAlpha = 0.6;
48
49 xlim([0, t1]);
50 y_max = max(abs([y; y_ref]));
51 ylim([-1.2*y_max, 1.2*y_max]);
52
53 xlabel('$t$', 'Interpreter', 'latex', 'FontSize', M)
54 ylabel('$y(t)$', 'Interpreter', 'latex', 'FontSize', M)
55 legend('$y(t)$', '$f(t)$', 'Interpreter', 'latex', 'Location', 'best', 'FontSize',
        'L');
56 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
57
58 %% график ошибки  $e(t) = y(t) - f(t)$ 
59 e = y - y_ref;
60
61 figure('Color','white','Position',[100 100 900 350]); hold on
62 plot(t, e, 'LineWidth', 2.5, 'Color', colors(1,:))
63
64 ax = gca;
65 ax.LineWidth = 0.8;
66 ax.FontSize = T;
67 ax.XColor = [0.3 0.3 0.3];
68 ax.YColor = [0.3 0.3 0.3];
69 ax.TickLength = [0 0];
70 ax.TickDir = 'out';
71

```

```

72 grid on;
73 box on;
74 ax.GridLineStyle = ':';
75 ax.GridColor = [0.7 0.7 0.7];
76 ax.GridAlpha = 0.6;
77
78 xlim([0, t1]);
79 e_max = max(abs(e));
80 if e_max < 1e-15
81     e_max = 1e-5;
82 end
83 ylim([-1.5*e_max, 1.5*e_max]);
84 ax.YAxis.Exponent = -3;
85
86 xlabel('$t$', 'Interpreter', 'latex', 'FontSize', M)
87 ylabel('$e(t)$', 'Interpreter', 'latex', 'FontSize', M)
88 legend('$e(t)=y(t)-f(t)$', 'Interpreter', 'latex', 'Location', 'best', 'FontSize', L);
89 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
    ;

```

Приложение Г

Листинг Г.1 — Нахождение матрицы наблюдаемости

```
1 % матрицы
2 A = [-16 -27 7; 6 9 -4; -5 -11 0];
3 C = [0 -5 -5];
4
5 % построение матрицы наблюдаемости V
6 V = [C; C*A; C*A^2];
7
8 % вывод результатов
9 disp('== Матрица наблюдаемости V ==');
10 disp('V =');
11 disp(V);
12
13 % находим ранг
14 rank_V = rank(V);
15 disp(['rank(V) = ', num2str(rank_V)]);
16
17 % проверка наблюдаемости
18 if rank_V == 3
19     disp('-> Система ПОЛНОСТЬЮ наблюдаема');
20 else
21     disp('-> Система НЕ ПОЛНОСТЬЮ наблюдаема');
22 end
```

Листинг Г.2 — Вычисление матриц Хаутуса для наблюдаемости

```
1 % матрицы
2 A = [-9 0 -10; -4 -1 -6; 6 -2 5];
3 C = [0 -5 -5];
4
5 % собственные числа
6 lambda = eig(A);
7
8 disp('Собственные числа матрицы A:');
9 disp(lambda);
10
11 n = size(A,1);
12
13 % массив для рангов
14 r = zeros(length(lambda),1);
15
16 for i = 1:length(lambda)
17
18     H = [lambda(i)*eye(n) - A; C];
19
20     fprintf('\nМатрица Хаутуса для lambda_%d:\n', i);
21     disp(H);
22 end
```

```

22
23     r(i) = rank(H);
24
25     fprintf('Ранг H_%d = %d\n', i, r(i));
26
27     if r(i) == n
28         disp('-> Собственное число наблюдаемо');
29     else
30         disp('-> Собственное число НЕ наблюдаемо');
31     end
32
33 end
34
35 % итоговый вывод по системе
36 disp('-----');
37
38 if all(r == n)
39     disp('СИСТЕМА ПОЛНОСТЬЮ НАБЛЮДАЕМА ПО КРИТЕРИЮ ХАУТУСА');
40 else
41     disp('СИСТЕМА НЕ ПОЛНОСТЬЮ НАБЛЮДАЕМА ПО КРИТЕРИЮ ХАУТУСА');
42 end

```

Листинг Г.3 — Вычисление Жордановой формы

```

1 % матрицы
2 A = [-9 0 -10; -4 -1 -6; 6 -2 5];
3 C = [0 -5 -5];
4
5 % собственные числа и векторы
6 [V, D] = eig(A);
7 lambda = diag(D); % столбец собственных чисел
8
9 disp('Собственные числа:');
10 disp(lambda);
11
12 % ищем комплексный и вещественный векторы
13 v_c = []; % комплексный вектор
14 v_r = []; % вещественный вектор
15
16 for k = 1:3
17     if abs(imag(lambda(k))) > 0.01 && isempty(v_c)
18         v_c = V(:, k); % комплексный вектор
19     elseif abs(imag(lambda(k))) < 0.01 && isempty(v_r)
20         v_r = real(V(:, k)); % вещественный вектор
21     end
22 end
23
24 % разбиваем комплексный вектор на вещественную и мнимую части
25 p1 = real(v_c);
26 p2 = imag(v_c);

```

```

27 p3 = v_r;
28
29 % матрица перехода P
30 P = [p1, p2, p3];
31
32 % проверяем, не вырождена ли P
33 if abs(det(P)) < 1e-10
34     disp('ВНИМАНИЕ: Матрица P вырождена! Меняем порядок столбцов...');
35     P = [p3, p1, p2]; % другой порядок
36 end
37
38 disp('Матрица P:'); disp(P);
39 disp('Определитель P:'); disp(det(P));
40
41 % обратная матрица
42 P_inv = inv(P);
43
44 % жорданова форма J = P^-1 * A * P
45 J = P_inv * A * P;
46
47 % матрица выхода в новом базисе
48 C_tilde = C * P;
49
50 % вывод результатов
51 disp('--- Результаты ---');
52 disp('Жорданова форма J:'); disp(J);
53 disp('Вектор C_tilde:'); disp(C_tilde);
54
55 % проверка наблюдаемости по строкам C_tilde
56 disp('--- Проверка наблюдаемости ---');
57 tolerance = 1e-6; % допуск
58
59 if abs(C_tilde(1)) > tolerance || abs(C_tilde(2)) > tolerance
60     disp('Блок J1 (комплексные корни) -> НАБЛЮДАЕМ');
61 else
62     disp('Блок J1 (комплексные корни) -> НЕ наблюдаем');
63 end
64
65 if abs(C_tilde(3)) > tolerance
66     disp('Блок J2 (вещественный корень) -> НАБЛЮДАЕМ');
67 else
68     disp('Блок J2 (вещественный корень) -> НЕ наблюдаем');
69 end

```

Листинг Г.4 — Вычисление Грамиана

```

1 % матрицы
2 A = [-9 0 -10; -4 -1 -6; 6 -2 5];
3 C = [0 -5 -5];
4 t1 = 3;

```

```

5
6 % подынтегральная функция: exp(A'*t) * C' * C * exp(A*t)
7 integrand = @(t) expm(A'*t) * C' * C * expm(A*t);
8
9 % численное интегрирование от 0 до t1
10 Q = integral(integrand, 0, t1, 'ArrayValued', true);
11
12 % вывод результатов
13 disp('== Грамиан наблюдаемости Q(t1) ==');
14 disp('Q =');
15 disp(Q);
16
17 % собственные числа
18 eig_Q = eig(Q);
19 disp('Собственные числа Грамиана:');
20 disp(eig_Q);
21
22 % анализ
23 if all(eig_Q > 0)
24     disp('-> Все собственные числа > 0: система ПОЛНОСТЬЮ наблюдаема');
25 else
26     disp('-> Внимание: есть нулевые или отрицательные числа');
27 end

```

Листинг Г.5 — Нахождение начальных условий

```

1 %% исходные данные
2 A = [-9 0 -10; -4 -1 -6; 6 -2 5];
3 C = [0 -5 -5];
4 t1 = 3;
5
6 f = @(t) -3*exp(-3*t).*cos(2*t) - 2*exp(-3*t).*sin(2*t);
7
8 %% вычисление Грамиана наблюдаемости
9 Q = integral(@(t) expm(A'*t)*C'*C*expm(A*t), 0, t1, 'ArrayValued', true);
10
11 %% вычисление начальных условий
12 I = integral(@(t) expm(A'*t)*C'*f(t), 0, t1, 'ArrayValued', true);
13 x0 = pinv(Q)*I; % Псевдообратная матрица (работает даже если Q вырожден)
14
15 disp('Начальные условия x(0):');
16 disp(x0);
17
18 %% решение ОДУ
19 tspan = linspace(0, t1, 1000);
20 [t, x] = ode45(@(t, x) A*x, tspan, x0);
21
22 %% вычисление выхода
23 y = (C*x')';
24 y_ref = f(t);

```

```

25
26 %% параметры оформления
27 colors = lines(7);
28 T = 12; M = 14; L = 12;
29
30 %% график  $y(t)$  и  $f(t)$ 
31 figure('Color','white','Position',[100 100 900 400]); hold on
32 plot(t, y, 'LineWidth', 2.5, 'Color', colors(2,:))
33 plot(t, y_ref, '--', 'LineWidth', 2.5, 'Color', colors(3,:))
34
35 ax = gca;
36 ax.LineWidth = 0.8;
37 ax.FontSize = T;
38 ax.XColor = [0.3 0.3 0.3];
39 ax.YColor = [0.3 0.3 0.3];
40 ax.TickLength = [0 0];
41 ax.TickDir = 'out';
42
43 grid on;
44 box on;
45 ax.GridLineStyle = ':';
46 ax.GridColor = [0.7 0.7 0.7];
47 ax.GridAlpha = 0.6;
48
49 xlim([0, t1]);
50 y_max = max(abs([y; y_ref]));
51 ylim([-1.2*y_max, 1.2*y_max]);
52
53 xlabel('$t$', 'Interpreter','latex','FontSize',M)
54 ylabel('$y(t)$', 'Interpreter','latex','FontSize',M)
55 legend('$y(t)$', '$f(t)$', 'Interpreter','latex', 'Location','best', 'FontSize',L);
56 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
57
58 %% график ошибки  $e(t) = y(t) - f(t)$ 
59 e = y - y_ref;
60
61 figure('Color','white','Position',[100 100 900 350]); hold on
62 plot(t, e, 'LineWidth', 2.5, 'Color', colors(1,:))
63
64 ax = gca;
65 ax.LineWidth = 0.8;
66 ax.FontSize = T;
67 ax.XColor = [0.3 0.3 0.3];
68 ax.YColor = [0.3 0.3 0.3];
69 ax.TickLength = [0 0];
70 ax.TickDir = 'out';
71

```

```

72 grid on;
73 box on;
74 ax.GridLineStyle = ':';
75 ax.GridColor = [0.7 0.7 0.7];
76 ax.GridAlpha = 0.6;
77
78 xlim([0, t1]);
79 e_max = max(abs(e));
80 if e_max < 1e-15
81     e_max = 1e-5;
82 end
83 ylim([-1.5*e_max, 1.5*e_max]);
84 ax.YAxis.Exponent = -3;
85
86 xlabel('$t$', 'Interpreter', 'latex', 'FontSize', M)
87 ylabel('$e(t)$', 'Interpreter', 'latex', 'FontSize', M)
88 legend('$e(t)=y(t)-f(t)$', 'Interpreter', 'latex', 'Location', 'best', 'FontSize', L);
89 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
90 ;

```

Листинг Г.6 — Нахождение альтернативных начальных условий

```

1 clear; clc; close all;
2
3 %% исходные данные
4 A = [-9 0 -10; -4 -1 -6; 6 -2 5];
5 C = [0 -5 -5];
6 t1 = 3;
7
8 f = @(t) -3*exp(-3*t).*cos(2*t) - 2*exp(-3*t).*sin(2*t);
9
10 %% вычисление Грамиана наблюдаемости
11 Q = integral(@(t) expm(A'*t)*C'*C*expm(A*t), 0, t1, 'ArrayValued', true);
12
13 %% вычисление начальных условий через псевдообратную матрицу
14 I = integral(@(t) expm(A'*t)*C'*f(t), 0, t1, 'ArrayValued', true);
15 x0_original = pinv(Q)*I; % Псевдообратная матрица
16
17 disp('=== Начальные условия x(0), вычисленные через Грамиан ===')
18 disp(x0_original)
19
20 %% находим ядро матрицы наблюдаемости
21 V = obsv(A, C);
22 null_V = null(V);
23
24 disp('=== Ядро матрицы наблюдаемости ===')
25 disp('Базисный вектор v:')
26 disp(null_V)
27

```



```

28 %% строим ЧЕТЫРЕ различных начальных условия
29 alpha1 = 0;
30 alpha2 = 1;
31 alpha3 = 2;
32 alpha4 = -1;
33
34 x0_1 = x0_original + alpha1 * null_V;
35 x0_2 = x0_original + alpha2 * null_V;
36 x0_3 = x0_original + alpha3 * null_V;
37 x0_4 = x0_original + alpha4 * null_V;
38
39 disp(' ')
40 disp('Четыре различных начальных условия:')
41 disp('x0_1 (исходное) ='); disp(x0_1)
42 disp('x0_2 ='); disp(x0_2)
43 disp('x0_3 ='); disp(x0_3)
44 disp('x0_4 ='); disp(x0_4)
45
46 %% моделирование
47 tspan = linspace(0, t1, 1000);
48
49 [t1_sol, x1_sol] = ode45(@(t,x) A*x, tspan, x0_1);
50 [t2_sol, x2_sol] = ode45(@(t,x) A*x, tspan, x0_2);
51 [t3_sol, x3_sol] = ode45(@(t,x) A*x, tspan, x0_3);
52 [t4_sol, x4_sol] = ode45(@(t,x) A*x, tspan, x0_4);
53
54 y1 = (C * x1_sol')';
55 y2 = (C * x2_sol')';
56 y3 = (C * x3_sol')';
57 y4 = (C * x4_sol')';
58 y_ref = f(tspan)';
59
60 %% ГРАФИКИ
61
62 %% рисунок 1 все выходы y(t)
63 figure('Color','white','Position',[100 100 900 400]); hold on
64
65 plot(tspan, y_ref, '-', 'LineWidth', 3, 'Color', [0 0.4470 0.7410 0.5])
66 plot(tspan, y1, '--', 'LineWidth', 3, 'Color', [0.8500 0.3250 0.0980 0.5])
67 plot(tspan, y2, ':', 'LineWidth', 3, 'Color', [0.4660 0.6740 0.1880 0.5])
68 plot(tspan, y3, '-.', 'LineWidth', 3, 'Color', [0.9290 0.6940 0.1250 0.5])
69 plot(tspan, y4, '--', 'LineWidth', 3, 'Color', [0.4940 0.1840 0.5560 0.5])
70
71 ax = gca;
72 ax.LineWidth = 0.8;
73 ax.FontSize = 12;
74 ax.XColor = [0.3 0.3 0.3];
75 ax.YColor = [0.3 0.3 0.3];
76 ax.TickLength = [0 0];

```

```

77
78 grid on; box on;
79 ax.GridLineStyle = ':';
80 ax.GridColor = [0.7 0.7 0.7];
81 ax.GridAlpha = 0.6;
82
83 xlim([0, t1]);
84
85 y_max = max([max(abs(y1)), max(abs(y2)), max(abs(y3)), ...
86             max(abs(y4)), max(abs(y_ref))]);
87 ylim([-1.2*y_max, 1.2*y_max]);
88
89 xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 14)
90 ylabel('$y(t)$', 'Interpreter', 'latex', 'FontSize', 14)
91 legend('$f(t)$', '$x_1(0)$', '$x_2(0)$', '$x_3(0)$', '$x_4(0)$', ...
92       'Interpreter', 'latex', 'Location', 'best', 'FontSize', 10);
93 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
94 ;
95 %% рисунок 2: компонента x_1(t)
96 figure('Color', 'white', 'Position', [100 100 900 400]); hold on
97 plot(tspan, x1_sol(:,1), 'LineWidth', 3, 'Color', [0 0.4470 0.7410])
98 plot(tspan, x2_sol(:,1), '--', 'LineWidth', 2.5, 'Color', [0.8500 0.3250
99     0.0980])
100 plot(tspan, x3_sol(:,1), ':', 'LineWidth', 2.5, 'Color', [0.9290 0.6940
101     0.1250])
102 plot(tspan, x4_sol(:,1), '-.', 'LineWidth', 2.5, 'Color', [0.4940 0.1840
103     0.5560])
104
105 ax = gca;
106 ax.LineWidth = 0.8;
107 ax.FontSize = 12;
108 ax.XColor = [0.3 0.3 0.3];
109 ax.YColor = [0.3 0.3 0.3];
110 ax.TickLength = [0 0];
111
112 grid on; box on;
113 ax.GridLineStyle = ':';
114 ax.GridColor = [0.7 0.7 0.7];
115 ax.GridAlpha = 0.6;
116
117 xlim([0, t1]);
118 xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 14)
119 ylabel('$x_1(t)$', 'Interpreter', 'latex', 'FontSize', 14)
120 legend('$x_1(0)$', '$x_2(0)$', '$x_3(0)$', '$x_4(0)$', ...
121       'Interpreter', 'latex', 'Location', 'best', 'FontSize', 10);
122 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
123 ;
124

```

```

121 %% рисунок 3: компонента  $x_2(t)$ 
122 figure('Color','white','Position',[100 100 900 400]); hold on
123 plot(tspan, x1_sol(:,2), 'LineWidth', 3, 'Color', [0 0.4470 0.7410])
124 plot(tspan, x2_sol(:,2), '--', 'LineWidth', 2.5, 'Color', [0.8500 0.3250
    0.0980])
125 plot(tspan, x3_sol(:,2), ':', 'LineWidth', 2.5, 'Color', [0.9290 0.6940
    0.1250])
126 plot(tspan, x4_sol(:,2), '-.', 'LineWidth', 2.5, 'Color', [0.4940 0.1840
    0.5560])
127
128 ax = gca;
129 ax.LineWidth = 0.8;
130 ax.FontSize = 12;
131 ax.XColor = [0.3 0.3 0.3];
132 ax.YColor = [0.3 0.3 0.3];
133 ax.TickLength = [0 0];
134
135 grid on; box on;
136 ax.GridLineStyle = ':';
137 ax.GridColor = [0.7 0.7 0.7];
138 ax.GridAlpha = 0.6;
139
140 xlim([0, t1]);
141 xlabel('$t$', 'Interpreter','latex','FontSize',14)
142 ylabel('$x_2(t)$', 'Interpreter','latex','FontSize',14)
143 legend('$x_1(0)$', '$x_2(0)$', '$x_3(0)$', '$x_4(0)$', ...
144     'Interpreter','latex', 'Location','best', 'FontSize',10);
145 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
146 ;
147
148 %% рисунок 4: компонента  $x_3(t)$ 
149 figure('Color','white','Position',[100 100 900 400]); hold on
150 plot(tspan, x1_sol(:,3), 'LineWidth', 3, 'Color', [0 0.4470 0.7410])
151 plot(tspan, x2_sol(:,3), '--', 'LineWidth', 2.5, 'Color', [0.8500 0.3250
    0.0980])
152 plot(tspan, x3_sol(:,3), ':', 'LineWidth', 2.5, 'Color', [0.9290 0.6940
    0.1250])
153 plot(tspan, x4_sol(:,3), '-.', 'LineWidth', 2.5, 'Color', [0.4940 0.1840
    0.5560])
154
155 ax = gca;
156 ax.LineWidth = 0.8;
157 ax.FontSize = 12;
158 ax.XColor = [0.3 0.3 0.3];
159 ax.YColor = [0.3 0.3 0.3];
160 ax.TickLength = [0 0];
161
162 grid on; box on;
163 ax.GridLineStyle = ':';

```

```

163 ax.GridColor = [0.7 0.7 0.7];
164 ax.GridAlpha = 0.6;
165
166 xlim([0, t1]);
167 xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 14)
168 ylabel('$x_3(t)$', 'Interpreter', 'latex', 'FontSize', 14)
169 legend('$x_1(0)$', '$x_2(0)$', '$x_3(0)$', '$x_4(0)$', ...
170        'Interpreter', 'latex', 'Location', 'best', 'FontSize', 10);
171 set(findall(gcf, '-property', 'FontName'), 'FontName', 'DejaVu Math TeX Gyre')
    ;

```

Приложение Д

Листинг Д.1 — Вычисление Жордановой формы

```
1 %% матрицы
2 A = [1 2 -1; -6 -7 -2; 6 6 1];
3 B = [-2; 4; -2];
4 C = [0 1 -1; 0 -3 3];
5
6 disp('=====');
7 disp('  ЖОРДАНОВА ФОРМА И АНАЛИЗ СИСТЕМЫ');
8 disp('=====');
9
10 %% собственные числа и векторы
11 [V, D] = eig(A);
12 lambda = diag(D);
13
14 disp(' ');
15 disp('Собственные числа:');
16 disp(lambda);
17
18 %% ищем комплексный и вещественный векторы
19 v_c = []; % комплексный вектор
20 v_r = []; % вещественный вектор
21
22 for k = 1:3
23     if abs(imag(lambda(k))) > 0.01 && isempty(v_c)
24         v_c = V(:, k); % комплексный вектор
25     elseif abs(imag(lambda(k))) < 0.01 && isempty(v_r)
26         v_r = real(V(:, k)); % вещественный вектор
27     end
28 end
29
30 %% разбиваем комплексный вектор на вещественную и мнимую части
31 p1 = real(v_c);
32 p2 = imag(v_c);
33 p3 = v_r;
34
35 %% матрица перехода P
36 P = [p1, p2, p3];
37
38 % проверяем, не вырождена ли P
39 if abs(det(P)) < 1e-10
40     disp('ВНИМАНИЕ: Матрица P вырождена! Меняем порядок столбцов...');
41     P = [p3, p1, p2]; % другой порядок
42 end
43
44 disp(' ');
45 disp('Матрица P:');
```

```

46 disp(P);
47 disp('Определитель P:');
48 disp(det(P));
49
50 %% обратная матрица
51 P_inv = inv(P);
52
53 %% жорданова форма J = P^-1 * A * P
54 J = P_inv * A * P;
55
56 %% вектор управления в новом базисе
57 B_tilde = P_inv * B;
58
59 %% матрица выхода в новом базисе
60 C_tilde = C * P;
61
62 %% вывод результатов
63 disp(' ');
64 disp('=====');
65 disp(' РЕЗУЛЬТАТЫ');
66 disp('=====');
67 disp(' ');
68 disp('Жорданова форма J:');
69 disp(J);
70 disp(' ');
71 disp('Вектор B_tilde:');
72 disp(B_tilde);
73 disp(' ');
74 disp('Матрица C_tilde:');
75 disp(C_tilde);
76
77 %% проверка управляемости
78 disp(' ');
79 disp('=====');
80 disp(' ПРОВЕРКА УПРАВЛЯЕМОСТИ');
81 disp('=====');
82
83 tolerance = 1e-6;
84
85 if abs(B_tilde(1)) > tolerance || abs(B_tilde(2)) > tolerance
86     disp('Блок J1 (комплексные корни) -> УПРАВЛЯЕМ');
87     controllable_1 = true;
88 else
89     disp('Блок J1 (комплексные корни) -> НЕ управляем');
90     controllable_1 = false;
91 end
92
93 if abs(B_tilde(3)) > tolerance
94     disp('Блок J2 (вещественный корень) -> УПРАВЛЯЕМ');

```

```

95     controllable_2 = true;
96 else
97     disp('Блок J2 (вещественный корень) -> НЕ управляем');
98     controllable_2 = false;
99 end
100
101 %% проверка наблюдаемости
102 disp(' ');
103 disp('=====');
104 disp('  ПРОВЕРКА НАБЛЮДАЕМОСТИ');
105 disp('=====');
106
107 % для блока J1 (2x2) проверяем первые две строки C_tilde
108 if any(abs(C_tilde(:,1:2)) > tolerance, 'all')
109     disp('Блок J1 (комплексные корни) -> НАБЛЮДАЕМ');
110     observable_1 = true;
111 else
112     disp('Блок J1 (комплексные корни) -> НЕ наблюдаем');
113     observable_1 = false;
114 end
115
116 % для блока J2 (1x1) проверяем третий столбец C_tilde
117 if any(abs(C_tilde(:,3)) > tolerance)
118     disp('Блок J2 (вещественный корень) -> НАБЛЮДАЕМ');
119     observable_2 = true;
120 else
121     disp('Блок J2 (вещественный корень) -> НЕ наблюдаем');
122     observable_2 = false;
123 end
124
125 %% общий вывод
126 disp(' ');
127 disp('=====');
128 disp('  ОБЩИЙ ВЫВОД');
129 disp('=====');
130
131 if controllable_1 && controllable_2
132     disp('СИСТЕМА ПОЛНОСТЬЮ УПРАВЛЯЕМА');
133 else
134     disp('СИСТЕМА НЕ ПОЛНОСТЬЮ УПРАВЛЯЕМА');
135 end
136
137 if observable_1 && observable_2
138     disp('СИСТЕМА ПОЛНОСТЬЮ НАБЛЮДАЕМА');
139 else
140     disp('СИСТЕМА НЕ ПОЛНОСТЬЮ НАБЛЮДАЕМА');
141 end
142
143 disp(' ');

```

Листинг Д.2 — Вычисление матрицы управляемости по выходу

```
1 % исходные данные
2 A = [1 2 -1; -6 -7 -2; 6 6 1];
3
4 B = [-2; 4; -2];
5
6 C = [0 1 -1; 0 -3 3];
7
8 D = zeros(2,1);
9
10 % матрица управляемости по выходу
11 Qy = [C*B, C*A*B, C*A^2*B];
12
13 disp('Матрица управляемости по выходу Qy =')
14 disp(Qy)
15
16 % ранг матрицы
17 r = rank(Qy);
18
19 fprintf('\nРанг матрицы: r = %d\n', r);
20
21 % число выходов
22 p = size(C,1);
23
24 if r == p
25     disp('Система управляема по выходу.')
26 else
27     disp('Система не управляема по выходу.')
28 end
```

Листинг Д.3 — MATLAB-скрипт исследования управляемости по выходу системы

```
1 A = [1 2 -1;
2      -6 -7 -2;
3      6 6 1];
4
5 B = [-2; 4; -2];
6
7 C = [0 1 -1;
8      0 -3 3];
9
10 %% D = 0 (исходный случай)
11 D1 = zeros(2,1);
12
13 Wy1 = [C*B, C*A*B, C*A^2*B];
14 r1 = rank(Wy1);
15
16 disp('D = 0')
17 disp(Wy1)
```



```

18 fprintf('rank = %d\n\n', r1);
19
20 %% D != 0 (улучшение)
21 D2 = [0; 1];
22
23 Wy2 = [D2, C*B, C*A*B, C*A^2*B];
24 r2 = rank(Wy2);
25
26 disp('D = [0;1]')
27 disp(Wy2)
28 fprintf('rank = %d\n', r2);
29
30 p = size(C,1);
31
32 if r2 == p
33     disp('Система стала полностью управляема по выходу')
34 else
35     disp('Система НЕ управляема по выходу')
36 end

```